REPÚBLICA DOMINICANA
UNIVERSIDAD AUTÓNOMA DE SANTO DOMINGO
FACULTAD DE CIENCIAS
ESCUELA DE INFORMÁTICA



Semestre: 2026-02
Asignatura: Base de Datos I
Clave y Sección: INF4200-5

Profesora: Rosmery Alberto Monegro

PROYECTO FINAL

Temas:
Sistema de Alquiler de Vehículos
"AutoDrive S.R.L."

Grupo 5:

Sustentantes:
11- Esteysi Chanell Martínez Batista **100583333**
13- Glimmer Wendy Méndez López **100193595**
17- Dodany de Jesús Pichardo Rodríguez **100558733**
Pedro Dulto Estanilao Alcalá **100570318**
27- Jorge Antonio Vásquez Peña **100437013**

Link de Acceso a Github: https://github.com/Glimer04/Proyecto_AutoDrive

Link de la Aplicación:
https://github.com/Glimer04/Proyecto_AutoDrive/blob/main/03_Aplicacion/sistema.py

04 de octubre 2026

2. Introducción

En el entorno empresarial actual, la gestión eficiente de la información es un factor determinante para la competitividad y la sostenibilidad de cualquier organización. El sector de alquiler de vehículos (rent-a-car) se caracteriza por manejar un alto volumen de transacciones diarias, donde la disponibilidad de la flota, la gestión de clientes y la facturación deben operar de manera sincronizada. La empresa AutoDrive S.R.L. enfrenta actualmente problemas derivados de la gestión manual de sus procesos, lo que genera riesgos de doble reserva, pérdida de información, lentitud en la facturación y dificultades para mantener un control exacto sobre el mantenimiento de sus vehículos.

Ante esta problemática, surge la necesidad de desarrollar e implementar el Sistema de Alquiler de Vehículos (SARV), una solución informática que centralice y automatice las operaciones del negocio. La correcta administración de los datos no solo facilita el trabajo diario de los agentes de ventas y administradores, sino que proporciona a la gerencia información confiable y en tiempo real para la toma de decisiones estratégicas.

Para el desarrollo de este proyecto, se utilizaron herramientas tecnológicas de estándar industrial. La base de datos relacional fue diseñada y estructurada en **Microsoft SQL Server**, aplicando conceptos de normalización, integridad referencial, y automatización mediante Triggers y Procedimientos Almacenados. Por otro lado, la interfaz gráfica y la lógica de conexión fueron desarrolladas en el lenguaje de programación **Python**, utilizando la librería **Tkinter** y el conector oficial **mssql_python**.

Con la culminación de este proyecto, se espera obtener como resultado un sistema funcional y robusto que elimine la redundancia de datos, garantice la integridad de la información y proporcione una experiencia ágil al usuario. Asimismo, este desarrollo representa la aplicación práctica e integradora de todos los conocimientos adquiridos durante la asignatura Base de Datos I, demostrando la capacidad del equipo para transformar un problema del mundo real en una solución tecnológica.

3. Índice temático

2. Introducción 2
3.1 Tabla de Ilustraciones 4
4. Descripción de la empresa 5
4.1. Descripción y naturaleza de la empresa 5
4.2. Misión, visión, valores y objetivos 6
4.3. Organigrama de la empresa 7
5. Descripción del Sistema de Información a analizar, diseñar y desarrollar 8
5.1. Descripción del Sistema de Información (SARV) 8
5.2. Diagrama de Flujo de Datos (DFD) 9
5.3. Narrativa de los procesos representados en los DFD 10
6. Modelo de Datos Conceptual 11
7. Modelo de Datos Lógico 12
8. Modelo de Datos Físico 14
9. Script de creación de la base de datos 14
10. Diccionario de Datos 21
11. Script de inserción de datos 23
12. Script de consultas 25
13. Vistas 29
14. Triggers y Stored Procedures 30
14.1. Triggers (Disparadores) 30
14.2. Stored Procedures (Procedimientos Almacenados) 35
15. Desarrollo de la aplicación 39
16. Conclusión General 42
17. Recomendaciones 43
18. Anexos 44

3.1 Tabla de Ilustraciones

Ilustración 1 - Organigrama (Estructura Organizacional) 7
Ilustración 2 - Diagrama DFD Nivel 0 (hecho en Draw.io) 9
Ilustración 3 - Diagrama DFD Nivel 1 (Hecho en Draw.io) 9
Ilustración 4 - Diagrama DFD Nivel 2 (hecho en Draw.io) 10
Ilustración 5 - Diagrama de Modelo Conceptual (hecho el Draw.io) 11
Ilustración 6 - Diagrama de Modelo Lógico (hecho en Draw.io) 13
Ilustración 7 - Diagrama de Modelo de Datos Físico (hecho en Draw.io) 14
Ilustración 8 - Consulta con INNER JOIN: Mostrar las reservas con nombre de cliente y placa del vehículo. 26
Ilustración 9 - Consulta con Funciones de Agregación: Total facturado por cliente. 27
Ilustración 10 - Consulta con GROUP BY y HAVING: Vehículos que han tenido más de 1 mantenimiento. 28
Ilustración 11 - Subconsulta: Clientes que no tienen ninguna reserva registrada. 29
Ilustración 12 - Caso de Prueba - Trigger.1 31
Ilustración 13 - Tabla de Auditoria 32
Ilustración 14 - Caso de Prueba - Trigger.2 33
Ilustración 15 - Caso de Prueba - SP.1 36
Ilustración 16 - Caso de Prueba - SP.2 37
Ilustración 17 - Aplicación Python: Login 40
Ilustración 18 - Menu del Administrador 40
Ilustración 19 - Menu del usuario 41
Ilustración 20 - Tabla de Vehiculos 41
Ilustración 21 - Tabla de Soporte Técnico 42
Ilustración 22 - Ejecucion Exitosa de la Creacion de las Tablas 44
Ilustración 23 - Listado de las Tablas Creadas en SQL Server 44
Ilustración 24 - Consulta de reservas con detalles de clientes y vehículos. 45
Ilustración 25 - Ejecución del procedimiento almacenado para el registro de reservas. 45
Ilustración 26 - Prueba de actualización de disponibilidad de vehículos mediante Trigger y Vista. 46
Ilustración 27 - Verificación del registro automático en la tabla de auditoría tras una modificación. 46
Ilustración 28 - Listado de vistas del sistema (Vista_ContratosActivos, Vista_EstadoFacturas, Vista_VehiculosDisponibles) en SQL Server. 47
Ilustración 29 - Estructura de disparadores (Triggers) implementados en las tablas del sistema. 47
Ilustración 30 - Estructura de procedimientos (Stored Procedures) almacenados creados en la sección de programabilidad. 48

4. Descripción de la empresa

4.1. Descripción y naturaleza de la empresa

AutoDrive S.R.L. es una sociedad de responsabilidad limitada constituida bajo las leyes comerciales de la República Dominicana, cuya actividad económica principal es la prestación de servicios de alquiler de vehículos de motor (rent-a-car). La empresa orienta sus operaciones hacia dos segmentos de mercado clave: el sector turístico, atendiendo a visitantes nacionales e internacionales que requieren movilidad durante su estancia en el país, y el sector corporativo, proveyendo soluciones de transporte a empresas que necesitan agilizar la movilidad de su personal o de sus invitados.

La naturaleza operativa de AutoDrive S.R.L. se fundamenta en un modelo de negocio integrado. La empresa no solo se limita a la entrega de vehículos, sino que gestiona un ciclo completo que abarca la reserva, el alquiler, la devolución y el mantenimiento. Para ello, cuenta con una flota diversificada que incluye vehículos tipo SUV, camionetas y compactos, adaptándose a las distintas necesidades de presupuesto y capacidad de sus clientes.

Un factor diferenciador y estratégico de la organización es su capacidad de integración vertical con un taller propio. Esta característica, poco común en las empresas del sector que suelen tercerizar el mantenimiento, permite a AutoDrive S.R.L. tener un control total sobre el estado mecánico y estético de su flota. Al gestionar internamente las inspecciones, reparaciones y el mantenimiento preventivo, la empresa garantiza una mayor disponibilidad de unidades, reduce los tiempos de inactividad y asegura que cada vehículo entregado cumpla con los más altos estándares de seguridad para el cliente.

En el ámbito tecnológico, AutoDrive S.R.L. reconoce que su crecimiento y la optimización de sus recursos dependen de la digitalización. Es por ello que la empresa ha iniciado la transición de sus procesos manuales hacia una gestión centralizada mediante el desarrollo de su Sistema de Alquiler de Vehículos (SARV). Este sistema está diseñado para interconectar todas las áreas de la empresa, desde la validación de la identidad y

licencia de un cliente, hasta el registro de la reserva, la facturación y la actualización del estado del vehículo en tiempo real, sentando las bases de una operación moderna, trazable y libre de errores operativos.

4.2. Misión, visión, valores y objetivos

Misión:

Proporcionar soluciones integrales de movilidad a turistas y al sector corporativo, garantizando la disponibilidad de una flota moderna y segura, respaldada por un mantenimiento técnico de excelencia y un servicio al cliente ágil. En AutoDrive S.R.L. nos comprometemos a optimizar la experiencia de alquiler mediante procesos eficientes y transparentes, asegurando que cada vehículo entregado cumpla con los más altos estándares de calidad y confort.

Visión:

Consolidarnos para el año 2030 como la empresa líder en el mercado de alquiler de vehículos en la República Dominicana, siendo reconocidos por nuestra innovación tecnológica, la eficiencia de nuestras operaciones y la confiabilidad de nuestro servicio. Aspiramos a transformar la experiencia tradicional de rent-a-car mediante la digitalización de nuestros procesos, la expansión estratégica de nuestra flota y la creación de alianzas que fortalezcan el turismo y la movilidad corporativa en el país.

Valores Corporativos:

- **Responsabilidad:** Actuamos con rigor y compromiso en la seguridad de nuestros clientes y en el cuidado de nuestra flota.
- **Transparencia:** Fomentamos una comunicación clara y honesta. Nuestros contratos, tarifas y políticas de alquiler están libres de letra pequeña.
- **Puntualidad:** Valoramos el tiempo de nuestros clientes. Nos esforzamos por cumplir estrictamente con los horarios de entrega y recepción.
- **Innovación:** Buscamos la mejora continua mediante la adopción de nuevas tecnologías (como el sistema SARV).

- **Vocación de Servicio:** Nos enfocamos en superar las expectativas del cliente, brindando una atención personalizada.

Objetivos:

1. Automatizar el proceso de reservas y disponibilidad de vehículos para evitar la doble reserva.
2. Integrar la gestión del taller mecánico con el estatus operativo de la flota.
3. Reducir los errores operativos en la facturación mediante un sistema centralizado.
   El sistema SARV contribuirá directamente a estos objetivos eliminando la gestión manual y agilizando los procesos de facturación y control de flota.

4.3. Organigrama de la empresa



Ilustración 1 - Organigrama (Estructura Organizacional)

Funciones de los departamentos:

- **Gerencia General:** Encargada de la toma de decisiones estratégicas y supervisión global.
- **Finanzas:** Administra los ingresos, facturación, pagos y presupuestos.

- **Operaciones (Flota/Taller):** Gestiona la disponibilidad de los vehículos y su mantenimiento preventivo y correctivo.
- **Ventas/Atención al Cliente:** Responsable de las reservas, contratos y atención directa al usuario.
- **Tecnología (TI):** Encargada del mantenimiento del sistema SARV, la base de datos y la seguridad informática.

5. Descripción del Sistema de Información a analizar, diseñar y desarrollar

5.1. Descripción del Sistema de Información (SARV)

El Sistema de Alquiler de Vehículos (SARV) es una solución informática diseñada para resolver la desorganización y los errores operativos derivados de la gestión manual. El sistema automatiza los módulos de: Seguridad y Empleados, Clientes, Flota, Operaciones, Facturación y Taller.

Requerimientos Funcionales:

- El sistema debe permitir el inicio de sesión diferenciando roles (Administrador, Agente de Ventas).
- Debe permitir registrar clientes, validando que su cédula y licencia sean únicas.
- Debe calcular automáticamente el costo del alquiler basándose en la tarifa diaria de la categoría del vehículo.
- Debe permitir registrar mantenimientos a los vehículos, cambiando su estado a "En Mantenimiento".

Requerimientos No Funcionales:

- **Seguridad:** Las contraseñas de los usuarios deben viajar de forma segura y el sistema debe bloquear accesos no autorizados.
- **Integridad:** La base de datos debe rechazar la inserción de reservas para fechas inválidas o clientes inexistentes.

- **Usabilidad:** La interfaz gráfica (desarrollada en Python/Tkinter) debe ser intuitiva y amigable.

5.2. Diagrama de Flujo de Datos (DFD)

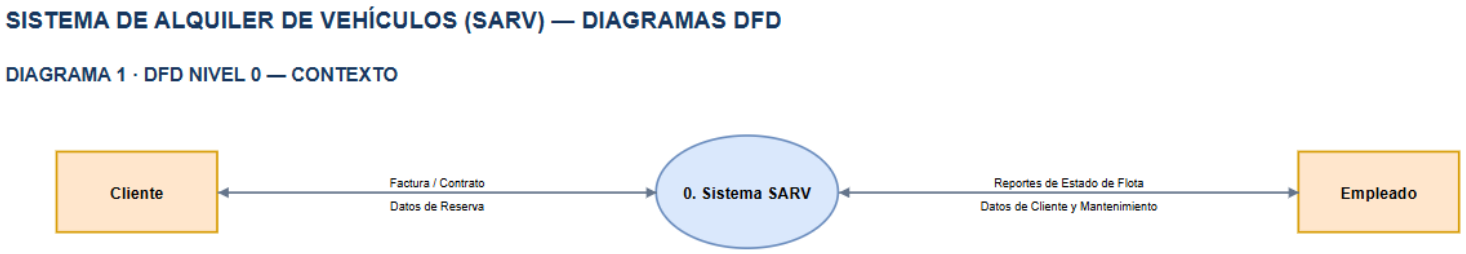


Ilustración 2 - Diagrama DFD Nivel 0 (hecho en Draw.io)

- **Nivel 0 (Contexto):** Un proceso central llamado `Sistema SARV`. Entidad externa izquierda: `Cliente` (envía datos de reserva). Entidad externa derecha: `Empleado` (envía datos de mantenimiento y recibe reportes).

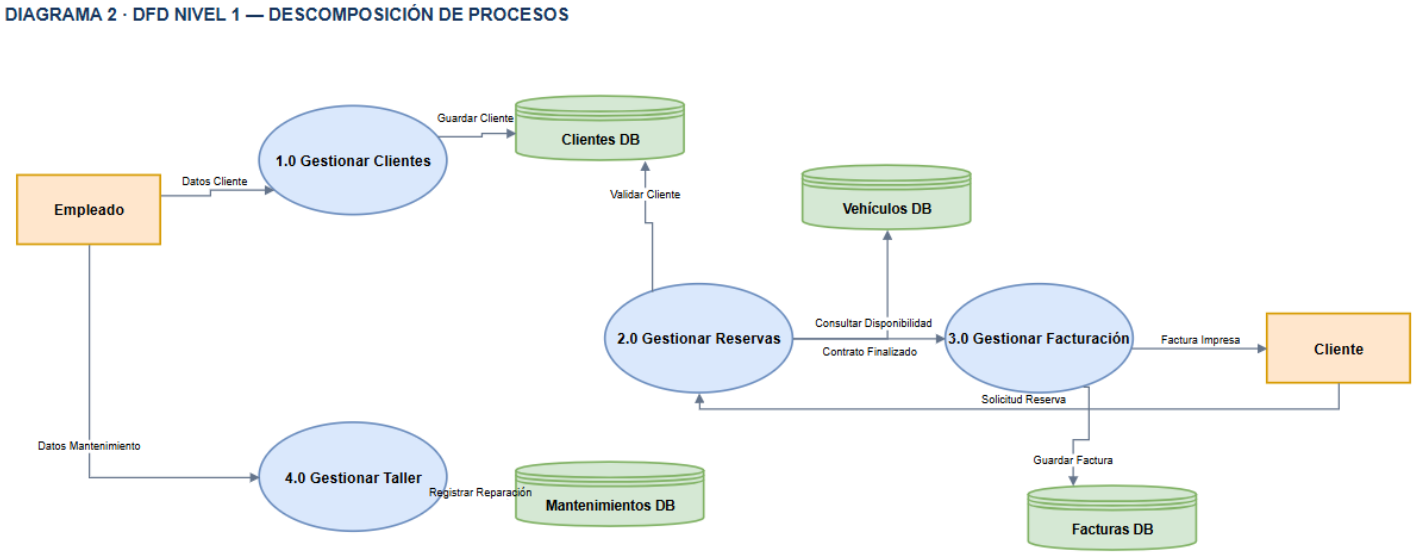


Ilustración 3 - Diagrama DFD Nivel 1 (Hecho en Draw.io)

- **Nivel 1:** Se descompone en: 1.0 Gestión de Clientes, 2.0 Gestión de Reservas, 3.0 Facturación y Pagos, 4.0 Gestión de Flota y Taller.

DIAGRAMA 3 · DFD NIVEL 2 — DETALLE DEL PROCESO 2.0 (RESERVAS)

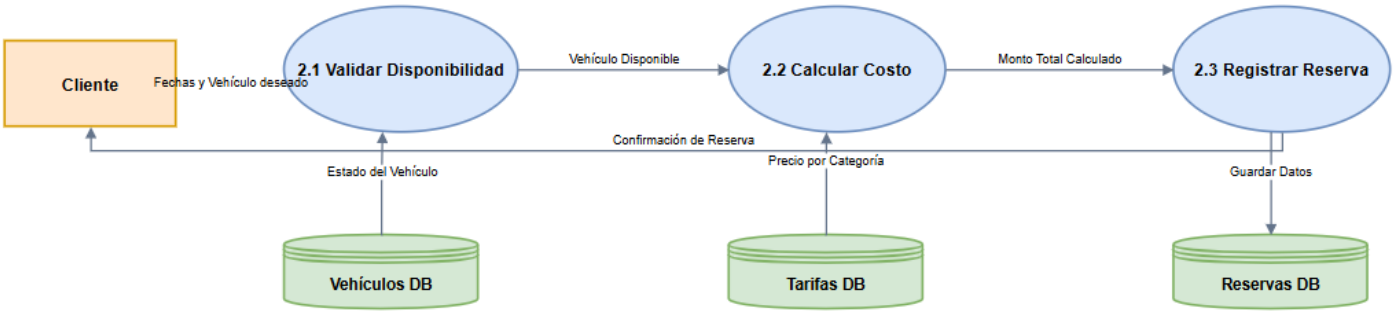


Ilustración 4 - Diagrama DFD Nivel 2 (hecho en Draw.io)

- **Nivel 2:** Desglosa el proceso 2.0 Gestión de Reservas en: 2.1 Validar disponibilidad del vehículo, 2.2 Registrar reserva en BD, 2.3 Generar contrato.

5.3. Narrativa de los procesos representados en los DFD

Proceso: Registrar Reserva

- **Objetivo:** Vincular a un cliente con un vehículo disponible por un periodo de tiempo.
- **Entrada:** Cédula del cliente, placa/categoría del vehículo, fechas de inicio y fin.
- **Actividades:** El sistema consulta la tabla Vehiculos y EstadosVehiculo. Si está "Disponible", procede a calcular el monto total multiplicando los días por la Tarifas_PrecioDiario.
- **Validaciones:** La fecha de fin debe ser estrictamente mayor a la de inicio (validado por el CHECK en base de datos y por el Stored Procedure).
- **Salida:** Registro en la tabla Reservas con estado "Pendiente" o "Confirmada".

6. Modelo de Datos Conceptual

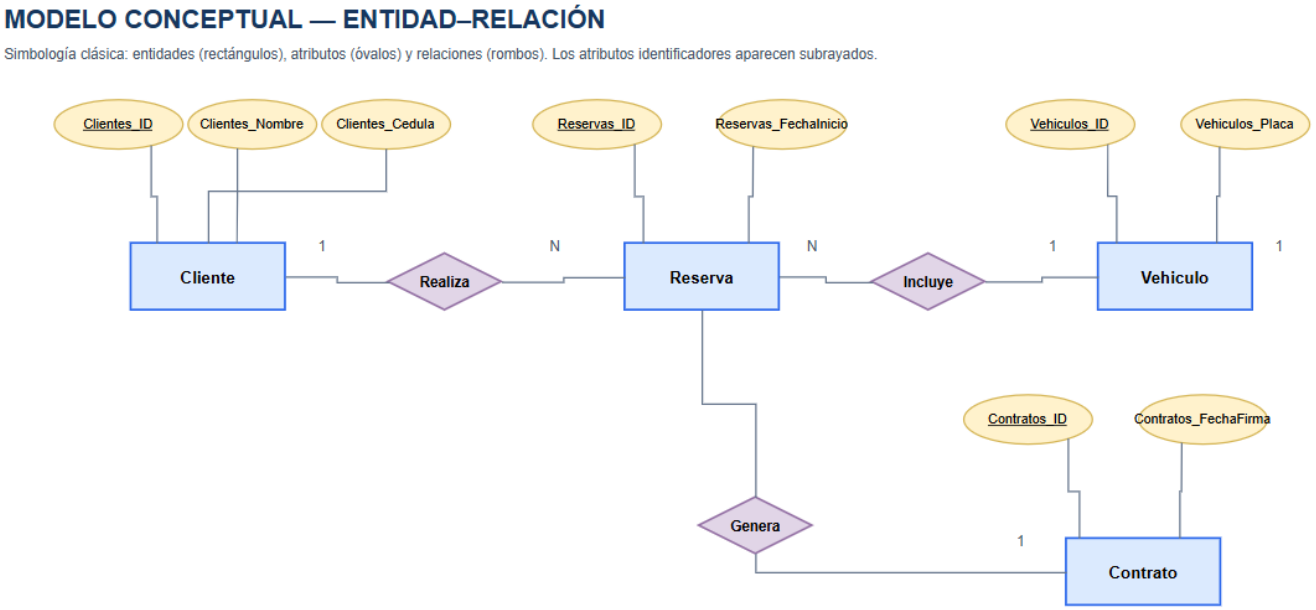


Ilustración 5 - Diagrama de Modelo Conceptual (hecho el Draw.io)

El modelo conceptual representa la lógica del negocio de forma independiente del gestor de base de datos que se vaya a utilizar. Para el sistema de **AutoDrive S.R.L.**, se ha diseñado un modelo Entidad-Relación (ER) robusto que abarca las 25 tablas requeridas, organizadas lógicamente en cinco módulos operativos. Esto garantiza que el sistema no tenga redundancias y cumpla con las reglas de normalización.

Las entidades principales y sus interrelaciones se estructuran de la siguiente manera:

A. Módulo de Clientes y Seguridad

- **Clientes:** Almacena los datos personales y de licencia del arrendatario.
- **Direcciones_Clientes:** Permite registrar múltiples direcciones por cliente.
- **Usuarios_Sistema y Roles:** Controlan el acceso de los empleados a la aplicación.

B. Módulo de Flota y Vehículos

- **Vehículos:** Entidad central que contiene la placa, modelo y estado actual del automóvil.
- **Categorías y Marcas:** Catálogos para clasificar los vehículos (SUV, compacto, etc.).
- **Seguros_Vehículo:** Registra las pólizas asociadas a cada unidad.
- **Estado_Flota:** Historial de disponibilidad (Disponible, Alquilado, En Taller).

C. Módulo de Reservas y Contratos

- **Reservas:** Vincula al cliente con un vehículo para un rango de fechas específico.
- **Contratos:** Formaliza legalmente la reserva una vez que el vehículo es entregado.
- **Detalle_Reserva y Cancelaciones:** Gestionan servicios adicionales y el historial de anulaciones.

D. Módulo de Facturación y Pagos

- **Facturas:** Documento fiscal generado por el contrato.
- **Detalle_Factura y Métodos_Pago:** Desglosan los cobros por alquiler, seguros y recargos.
- **Impuestos:** Catálogo de impuestos aplicables (ej. ITBIS).

E. Módulo de Mantenimiento y Taller

- **Mantenimientos:** Registra las entradas del vehículo al taller propio.
- **Detalle_Mantenimiento:** Especifica las reparaciones realizadas y repuestos utilizados.
- **Proveedores_Repuestos:** Registra a los suplidores de piezas.

7. Modelo de Datos Lógico

MODELO LÓGICO — TABLAS Y CLAVES

Tablas relacionales: PK = clave primaria · FK = clave foránea. Las líneas representan las referencias entre claves.

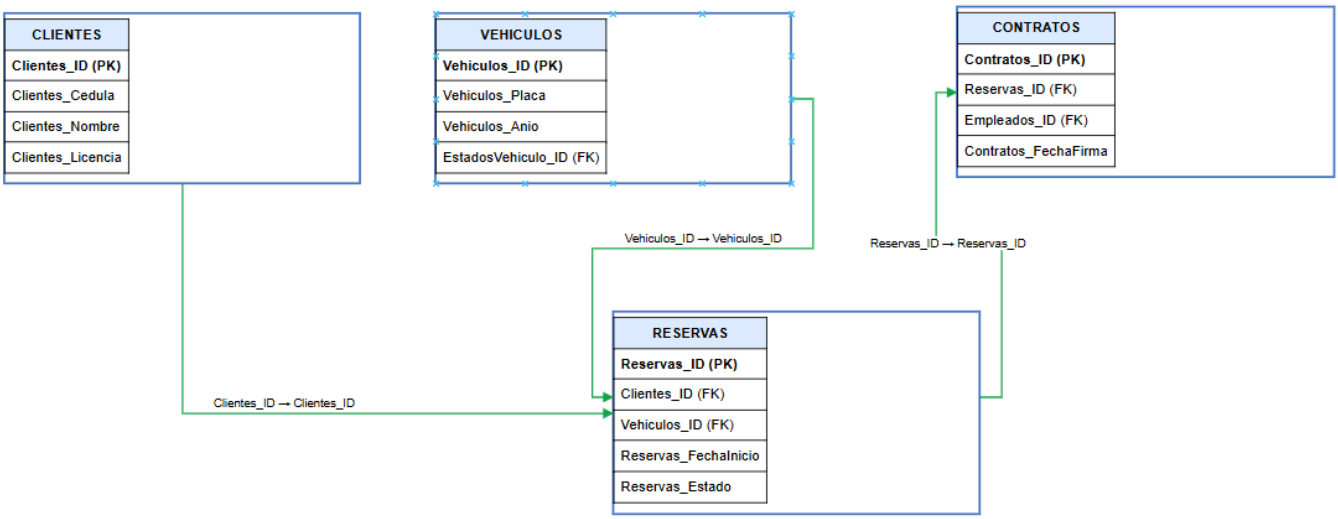


Ilustración 6 - Diagrama de Modelo Lógico (hecho en Draw.io)

- **Clientes**: Clientes_ID (PK), Clientes_Cedula, Clientes_Nombre, Clientes_Licencia.
- **Vehiculos**: Vehiculos_ID (PK), Vehiculos_Placa, Vehiculos_Anio, Marcas_ID (FK), Categorias_ID (FK), EstadosVehiculo_ID (FK).
- **Reservas**: Reservas_ID (PK), Clientes_ID (FK), Vehiculos_ID (FK), Reservas_FechaInicio, Reservas_FechaFin.
- **Contratos**: Contratos_ID (PK), Reservas_ID (FK), Empleados_ID (FK).
- **Facturas**: Facturas_ID (PK), Contratos_ID (FK), Facturas_MontoTotal.

8. Modelo de Datos Físico

MODELO FÍSICO — SQL SERVER

Definición física con tres columnas: Nombre del Campo | Tipo de Dato | Restricciones.

CLIENTES		
Nombre del Campo	**Tipo de Dato**	**Restricciones**
Clientes_ID	INT IDENTITY	PK
Clientes_Cedula	VARCHAR(15)	NOT NULL UNIQUE
Clientes_Nombre	VARCHAR(100)	NOT NULL

VEHICULOS		
Nombre del Campo	**Tipo de Dato**	**Restricciones**
Vehiculos_ID	INT IDENTITY	PK
Vehiculos_Placa	VARCHAR(10)	NOT NULL UNIQUE
EstadosVehiculo_ID	INT	FK

RESERVAS		
Nombre del Campo	**Tipo de Dato**	**Restricciones**
Reservas_ID	INT IDENTITY	PK
Clientes_ID	INT	FK
Reservas_FechaInicio	DATE	NOT NULL
Reservas_Estado	VARCHAR(20)	DEFAULT 'Pendiente'

CONTRATOS		
Nombre del Campo	**Tipo de Dato**	**Restricciones**
Contratos_ID	INT IDENTITY	PK
Reservas_ID	INT	FK UNIQUE
Contratos_FechaFirma	DATETIME	DEFAULT GETDATE()

Ilustración 7 - Diagrama de Modelo de Datos Físico (hecho en Draw.io)

- **Clientes**: `Clientes_ID` (INT IDENTITY), `Clientes_Cedula` (VARCHAR(15)), `Clientes_Nombre` (VARCHAR(100)).
- **Reservas**: `Reservas_ID` (INT IDENTITY), `Clientes_ID` (INT), `Reservas_FechaInicio` (DATE), `Reservas_Estado` (VARCHAR(20) DEFAULT 'Pendiente').
- **Facturas**: `Facturas_MontoTotal` (DECIMAL(10,2)), `Facturas_FechaEmision` (DATETIME DEFAULT GETDATE()).

9. Script de creación de la base de datos

A continuación, se presenta el script DDL (Data Definition Language) que crea la base de datos `AutoDriveDB` con sus 25 tablas, aplicando la nomenclatura solicitada, tipos de datos, llaves primarias, foráneas, restricciones `CHECK`, valores `DEFAULT` e índices.

```
-- ============================================
-- PROYECTO FINAL: AUTODRIVE S.R.L.
-- SCRIPT DE CREACIÓN DE BASE DE DATOS (DDL)
-- ============================================
```

```sql
CREATE DATABASE AutoDriveDB;
GO

USE AutoDriveDB;
GO

-- 1. MÓDULO DE SEGURIDAD Y EMPLEADOS
CREATE TABLE Roles (
    Roles_ID INT IDENTITY(1,1) PRIMARY KEY,
    Roles_Nombre VARCHAR(50) NOT NULL UNIQUE
);
GO

CREATE TABLE Usuarios (
    Usuarios_ID INT IDENTITY(1,1) PRIMARY KEY,
    Usuarios_Nombre VARCHAR(50) NOT NULL UNIQUE,
    Usuarios_Clave VARCHAR(255) NOT NULL,
    Roles_ID INT NOT NULL,
    Usuarios_Estado VARCHAR(20) DEFAULT 'Activo' CHECK (Usuarios_Estado IN ('Activo', 'Inactivo')),
    CONSTRAINT FK_Usuarios_Roles FOREIGN KEY (Roles_ID) REFERENCES Roles(Roles_ID)
);
GO

CREATE TABLE Empleados (
    Empleados_ID INT IDENTITY(1,1) PRIMARY KEY,
    Empleados_Cedula VARCHAR(15) NOT NULL UNIQUE,
    Empleados_Nombre VARCHAR(100) NOT NULL,
    Empleados_Cargo VARCHAR(50) NOT NULL,
    Empleados_FechaIngreso DATE DEFAULT GETDATE()
);
GO

-- 2. MÓDULO DE CLIENTES
CREATE TABLE Clientes (
    Clientes_ID INT IDENTITY(1,1) PRIMARY KEY,
    Clientes_Cedula VARCHAR(15) NOT NULL UNIQUE,
    Clientes_Nombre VARCHAR(100) NOT NULL,
    Clientes_Licencia VARCHAR(20) NOT NULL UNIQUE,
    Clientes_FechaRegistro DATETIME DEFAULT GETDATE()
);
GO

CREATE TABLE Direcciones (
    Direcciones_ID INT IDENTITY(1,1) PRIMARY KEY,
```

```
    Clientes_ID INT NOT NULL,
    Direcciones_Detalle VARCHAR(150) NOT NULL,
    Direcciones_Ciudad VARCHAR(50) NOT NULL,
    CONSTRAINT FK_Direcciones_Clientes FOREIGN KEY (Clientes_ID) REFERENCES Clientes(Clientes_ID)
ON DELETE CASCADE
);
GO

CREATE TABLE Contactos (
    Contactos_ID INT IDENTITY(1,1) PRIMARY KEY,
    Clientes_ID INT NOT NULL,
    Contactos_Telefono VARCHAR(15) NOT NULL,
    Contactos_Email VARCHAR(100) CHECK (Contactos_Email LIKE '%@%.%'),
    CONSTRAINT FK_Contactos_Clientes FOREIGN KEY (Clientes_ID) REFERENCES Clientes(Clientes_ID) ON
DELETE CASCADE
);
GO

-- 3. MÓDULO DE FLOTA
CREATE TABLE Marcas (
    Marcas_ID INT IDENTITY(1,1) PRIMARY KEY,
    Marcas_Nombre VARCHAR(50) NOT NULL UNIQUE
);
GO

CREATE TABLE Categorias (
    Categorias_ID INT IDENTITY(1,1) PRIMARY KEY,
    Categorias_Nombre VARCHAR(50) NOT NULL UNIQUE,
    Categorias_Descripcion VARCHAR(150)
);
GO

CREATE TABLE EstadosVehiculo (
    EstadosVehiculo_ID INT IDENTITY(1,1) PRIMARY KEY,
    EstadosVehiculo_Descripcion VARCHAR(30) NOT NULL UNIQUE
);
GO

CREATE TABLE Vehiculos (
    Vehiculos_ID INT IDENTITY(1,1) PRIMARY KEY,
    Vehiculos_Placa VARCHAR(10) NOT NULL UNIQUE,
    Vehiculos_Anio INT NOT NULL CHECK (Vehiculos_Anio >= 2000),
    Marcas_ID INT NOT NULL,
    Categorias_ID INT NOT NULL,
    EstadosVehiculo_ID INT NOT NULL,
```

```
    CONSTRAINT FK_Vehiculos_Marcas FOREIGN KEY (Marcas_ID) REFERENCES Marcas(Marcas_ID),
    CONSTRAINT FK_Vehiculos_Categorias FOREIGN KEY (Categorias_ID) REFERENCES
Categorias(Categorias_ID),
    CONSTRAINT FK_Vehiculos_Estados FOREIGN KEY (EstadosVehiculo_ID) REFERENCES
EstadosVehiculo(EstadosVehiculo_ID)
);
GO

CREATE TABLE Tarifas (
    Tarifas_ID INT IDENTITY(1,1) PRIMARY KEY,
    Categorias_ID INT NOT NULL,
    Tarifas_PrecioDiario DECIMAL(10,2) NOT NULL CHECK (Tarifas_PrecioDiario > 0),
    CONSTRAINT FK_Tarifas_Categorias FOREIGN KEY (Categorias_ID) REFERENCES
Categorias(Categorias_ID)
);
GO

CREATE TABLE Seguros (
    Seguros_ID INT IDENTITY(1,1) PRIMARY KEY,
    Vehiculos_ID INT NOT NULL,
    Seguros_Poliza VARCHAR(50) NOT NULL,
    Seguros_Costo DECIMAL(10,2) NOT NULL CHECK (Seguros_Costo >= 0),
    CONSTRAINT FK_Seguros_Vehiculos FOREIGN KEY (Vehiculos_ID) REFERENCES Vehiculos(Vehiculos_ID)
);
GO

-- 4. MÓDULO DE OPERACIONES
CREATE TABLE Reservas (
    Reservas_ID INT IDENTITY(1,1) PRIMARY KEY,
    Clientes_ID INT NOT NULL,
    Vehiculos_ID INT NOT NULL,
    Reservas_FechaInicio DATE NOT NULL,
    Reservas_FechaFin DATE NOT NULL,
    Reservas_Estado VARCHAR(20) DEFAULT 'Pendiente' CHECK (Reservas_Estado IN ('Pendiente',
'Confirmada', 'Cancelada')),
    CONSTRAINT FK_Reservas_Clientes FOREIGN KEY (Clientes_ID) REFERENCES Clientes(Clientes_ID),
    CONSTRAINT FK_Reservas_Vehiculos FOREIGN KEY (Vehiculos_ID) REFERENCES
Vehiculos(Vehiculos_ID),
    CONSTRAINT CHK_Fechas CHECK (Reservas_FechaFin > Reservas_FechaInicio)
);
GO

CREATE TABLE Contratos (
    Contratos_ID INT IDENTITY(1,1) PRIMARY KEY,
    Reservas_ID INT NOT NULL UNIQUE,
```

```
    Empleados_ID INT NOT NULL,
    Contratos_FechaFirma DATETIME DEFAULT GETDATE(),
    Contratos_Estado VARCHAR(20) DEFAULT 'Activo' CHECK (Contratos_Estado IN ('Activo', 'Cerrado',
'Anulado')),
    CONSTRAINT FK_Contratos_Reservas FOREIGN KEY (Reservas_ID) REFERENCES
Reservas(Reservas_ID),
    CONSTRAINT FK_Contratos_Empleados FOREIGN KEY (Empleados_ID) REFERENCES
Empleados(Empleados_ID)
);
GO

CREATE TABLE Entregas (
    Entregas_ID INT IDENTITY(1,1) PRIMARY KEY,
    Contratos_ID INT NOT NULL,
    Entregas_Kilometraje INT NOT NULL CHECK (Entregas_Kilometraje >= 0),
    Entregas_NivelCombustible VARCHAR(20) NOT NULL,
    Entregas_Observaciones VARCHAR(255),
    CONSTRAINT FK_Entregas_Contratos FOREIGN KEY (Contratos_ID) REFERENCES
Contratos(Contratos_ID)
);
GO

CREATE TABLE Devoluciones (
    Devoluciones_ID INT IDENTITY(1,1) PRIMARY KEY,
    Contratos_ID INT NOT NULL UNIQUE,
    Devoluciones_Fecha DATETIME DEFAULT GETDATE(),
    Devoluciones_Kilometraje INT NOT NULL CHECK (Devoluciones_Kilometraje >= 0),
    CONSTRAINT FK_Devoluciones_Contratos FOREIGN KEY (Contratos_ID) REFERENCES
Contratos(Contratos_ID)
);
GO

CREATE TABLE Multas (
    Multas_ID INT IDENTITY(1,1) PRIMARY KEY,
    Devoluciones_ID INT NOT NULL,
    Multas_Motivo VARCHAR(150) NOT NULL,
    Multas_Monto DECIMAL(10,2) NOT NULL CHECK (Multas_Monto > 0),
    CONSTRAINT FK_Multas_Devoluciones FOREIGN KEY (Devoluciones_ID) REFERENCES
Devoluciones(Devoluciones_ID)
);
GO

-- 5. MÓDULO DE FACTURACIÓN
CREATE TABLE Facturas (
    Facturas_ID INT IDENTITY(1,1) PRIMARY KEY,
```

```
    Contratos_ID INT NOT NULL UNIQUE,
    Facturas_FechaEmision DATETIME DEFAULT GETDATE(),
    Facturas_MontoTotal DECIMAL(10,2) NOT NULL CHECK (Facturas_MontoTotal >= 0),
    Facturas_Estado VARCHAR(20) DEFAULT 'Pendiente' CHECK (Facturas_Estado IN ('Pendiente', 'Pagada',
'Anulada')),
    CONSTRAINT FK_Facturas_Contratos FOREIGN KEY (Contratos_ID) REFERENCES Contratos(Contratos_ID)
);
GO

CREATE TABLE MetodosPago (
    MetodosPago_ID INT IDENTITY(1,1) PRIMARY KEY,
    MetodosPago_Descripcion VARCHAR(50) NOT NULL UNIQUE
);
GO

CREATE TABLE Pagos (
    Pagos_ID INT IDENTITY(1,1) PRIMARY KEY,
    Facturas_ID INT NOT NULL,
    MetodosPago_ID INT NOT NULL,
    Pagos_Monto DECIMAL(10,2) NOT NULL CHECK (Pagos_Monto > 0),
    Pagos_Fecha DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_Pagos_Facturas FOREIGN KEY (Facturas_ID) REFERENCES Facturas(Facturas_ID),
    CONSTRAINT FK_Pagos_Metodos FOREIGN KEY (MetodosPago_ID) REFERENCES
MetodosPago(MetodosPago_ID)
);
GO

-- 6. MÓDULO DE TALLER Y SOPORTE (Para llegar a las 24 tablas)
CREATE TABLE Proveedores (
    Proveedores_ID INT IDENTITY(1,1) PRIMARY KEY,
    Proveedores_Nombre VARCHAR(100) NOT NULL,
    Proveedores_Telefono VARCHAR(15),
    Proveedores_Email VARCHAR(100)
);
GO

CREATE TABLE Mantenimientos (
    Mantenimientos_ID INT IDENTITY(1,1) PRIMARY KEY,
    Vehiculos_ID INT NOT NULL,
    Proveedores_ID INT NOT NULL,
    Mantenimientos_Fecha DATE DEFAULT GETDATE(),
    Mantenimientos_CostoTotal DECIMAL(10,2) DEFAULT 0 CHECK (Mantenimientos_CostoTotal >= 0),
    CONSTRAINT FK_Mantenimientos_Vehiculos FOREIGN KEY (Vehiculos_ID) REFERENCES
Vehiculos(Vehiculos_ID),
```

```
    CONSTRAINT FK_Mantenimientos_Proveedores FOREIGN KEY (Proveedores_ID) REFERENCES
Proveedores(Proveedores_ID)
);
GO

CREATE TABLE DetalleMantenimiento (
    DetalleMantenimiento_ID INT IDENTITY(1,1) PRIMARY KEY,
    Mantenimientos_ID INT NOT NULL,
    DetalleMantenimiento_Descripcion VARCHAR(150) NOT NULL,
    DetalleMantenimiento_Costo DECIMAL(10,2) NOT NULL CHECK (DetalleMantenimiento_Costo >= 0),
    CONSTRAINT FK_Detalle_Mantenimientos FOREIGN KEY (Mantenimientos_ID) REFERENCES
Mantenimientos(Mantenimientos_ID) ON DELETE CASCADE
);
GO

CREATE TABLE Auditoria (
    Auditoria_ID INT IDENTITY(1,1) PRIMARY KEY,
    Auditoria_Tabla VARCHAR(50) NOT NULL,
    Auditoria_Operacion VARCHAR(20) NOT NULL,
    Auditoria_Fecha DATETIME DEFAULT GETDATE(),
    Auditoria_Usuario VARCHAR(50) DEFAULT SYSTEM_USER,
    Auditoria_Detalle VARCHAR(255)
);
GO

CREATE TABLE SolicitudesSoporte (
    SolicitudesSoporte_ID INT IDENTITY(1,1) PRIMARY KEY,
    SolicitudesSoporte_Nombre VARCHAR(100) NOT NULL,
    SolicitudesSoporte_Email VARCHAR(100) NOT NULL,
    SolicitudesSoporte_Mensaje VARCHAR(500) NOT NULL,
    SolicitudesSoporte_Fecha DATETIME DEFAULT GETDATE(),
    SolicitudesSoporte_Estado VARCHAR(20) DEFAULT 'Pendiente'
);
GO

-- CREACIÓN DE ÍNDICES
CREATE INDEX IDX_Clientes_Cedula ON Clientes(Clientes_Cedula);
CREATE INDEX IDX_Vehiculos_Placa ON Vehiculos(Vehiculos_Placa);
CREATE INDEX IDX_Reservas_Cliente ON Reservas(Clientes_ID);
CREATE INDEX IDX_Reservas_Vehiculo ON Reservas(Vehiculos_ID);
CREATE INDEX IDX_Contratos_Reserva ON Contratos(Reservas_ID);
CREATE INDEX IDX_Pagos_Factura ON Pagos(Facturas_ID);
GO
```

10. Diccionario de Datos

El diccionario de datos es la documentación técnica central del sistema, ya que describe detalladamente la estructura lógica y física de la base de datos. En este apartado se especifica, para cada tabla y campo, su nombre, tipo de dato, longitud, si permite o no valores nulos (Nulabilidad), su rol como clave (PK o FK), los valores por defecto (DEFAULT) y las restricciones aplicadas (CHECK o UNIQUE).

Debido a la magnitud del proyecto (25 tablas en total), a continuación, se presenta un **extracto representativo** del diccionario de datos físico, abarcando las tablas principales del núcleo del negocio: *Clientes, Vehículos, Reservas y Contratos*. *(Nota: El diccionario de datos completo, con la documentación de las 25 tablas, se encuentra disponible en el repositorio de GitHub y en los anexos de este proyecto).*

Tabla: Clientes

Campo	Descripción	Tipo de Dato	Longitud	Nulabilidad	Clave	Default	Restricciones
Clientes_ID	Identificador único	INT	-	NOT NULL	PK	IDENTITY	-
Clientes_Cedula	Documento de identidad	VARCHAR	15	NOT NULL	UNIQUE	-	-
Clientes_Nombre	Nombre completo	VARCHAR	100	NOT NULL	-	-	-
Clientes_Licencia	Número de licencia	VARCHAR	20	NOT NULL	UNIQUE	-	-
Clientes_FechaRegistro	Fecha de alta	DATETIME	-	NULL	-	GETDATE()	-

Tabla: Vehiculos

Campo	Descripción	Tipo de Dato	Longitud	Nulabilidad	Clave	Default	Restricciones
Vehiculos_ID	Identificador único	INT	-	NOT NULL	PK	IDENTITY	-
Vehiculos_Placa	Placa del vehículo	VARCHAR	10	NOT NULL	UNIQUE	-	-
Vehiculos_Anio	Año de fabricación	INT	-	NOT NULL	-	-	>= 2000
Marcas_ID	FK de la marca	INT	-	NOT NULL	FK	-	Ref. Marcas
Categorias_ID	FK de la categoría	INT	-	NOT NULL	FK	-	Ref. Categorias
EstadosVehiculo_ID	Estado actual	INT	-	NOT NULL	FK	-	Ref. Estados

Tabla: Reservas

Campo	Descripción	Tipo de Dato	Longitud	Nulabilidad	Clave	Default	Restricciones
Reservas_ID	Identificador de reserva	INT	-	NOT NULL	PK	IDENTITY	-
Clientes_ID	FK del cliente	INT	-	NOT NULL	FK	-	Ref. Clientes
Vehiculos_ID	FK del vehículo	INT	-	NOT NULL	FK	-	Ref. Vehiculos
Reservas_FechaInicio	Fecha de inicio	DATE	-	NOT NULL	-	-	-
Reservas_FechaFin	Fecha de fin	DATE	-	NOT NULL	-	-	> FechaInicio
Reservas_Estado	Estado de reserva	VARCHAR	20	NULL	-	Pendiente	Lista de valores

Tabla: Contratos

Campo	Descripción	Tipo de Dato	Longitud	Nulabilidad	Clave	Default	Restricciones
Contratos_ID	Identificador único del contrato	INT	-	NOT NULL	PK	IDENTITY	-
Reservas_ID	FK de la reserva asociada	INT	-	NOT NULL	UNIQUE (FK)	-	Ref. Reservas
Empleados_ID	FK del empleado que firma	INT	-	NOT NULL	FK	-	Ref. Empleados
Contratos_FechaFirma	Fecha en que se firmó el contrato	DATETIME	-	NULL	-	GETDATE()	-
Contratos_Estado	Estado actual del contrato	VARCHAR	20	NULL	-	'Activo'	Lista de valores (Activo, Cerrado, Anulado)

11. Script de inserción de datos

El presente script de Lenguaje de Manipulación de Datos (DML) tiene como objetivo poblar la base de datos `AutoDriveDB` con un conjunto de datos de prueba coherentes con la lógica del negocio de AutoDrive S.R.L.

La inserción de los datos se ha estructurado respetando estrictamente el orden de dependencia de las llaves foráneas (Foreign Keys) para garantizar la integridad referencial. En primer lugar, se insertan los datos de las tablas maestras e independientes (Roles, Empleados, Clientes, Marcas, Categorías y Métodos de Pago). Posteriormente, se insertan los registros en las tablas transaccionales y dependientes (Vehículos, Reservas, Contratos, Facturas y Pagos).

Estos datos de prueba permiten validar el correcto funcionamiento de las consultas, vistas, triggers y procedimientos almacenados desarrollados en los apartados anteriores, simulando escenarios reales de alquiler de vehículos.

```
USE AutoDriveDB;
GO

INSERT INTO Roles (Roles_Nombre) VALUES ('Administrador'), ('Agente de Ventas');
INSERT INTO Usuarios (Usuarios_Nombre, Usuarios_Clave, Roles_ID) VALUES ('admin', '12345', 1),
('juan_agente', '12345', 2);
INSERT INTO Empleados (Empleados_Cedula, Empleados_Nombre, Empleados_Cargo) VALUES
('001-1111111-1', 'Carlos Pérez', 'Gerente'), ('001-2222222-2', 'Ana Gómez', 'Agente de Alquiler');
INSERT INTO Clientes (Clientes_Cedula, Clientes_Nombre, Clientes_Licencia) VALUES ('402-
1234567-8', 'Luis Martínez', 'LIC-99881'), ('402-8765432-1', 'María Rodríguez', 'LIC-77662'), ('001-
5555555-5', 'Empresa XYZ S.R.L.', 'LIC-CORP01');

INSERT INTO Contactos (Clientes_ID, Contactos_Telefono, Contactos_Email) VALUES (1, '809-555-
1234', 'luis@gmail.com'), (2, '829-555-5678', 'maria@hotmail.com'), (3, '809-555-0000',
'logistica@xyz.com');
INSERT INTO Direcciones (Clientes_ID, Direcciones_Detalle, Direcciones_Ciudad) VALUES (1, 'Calle
1 #23, Ensanche Naco', 'Santo Domingo'), (2, 'Av. Estrella Sadhalá #45', 'Santiago'), (3, 'Av. 27 de
Febrero #100', 'Santo Domingo');

INSERT INTO Marcas (Marcas_Nombre) VALUES ('Toyota'), ('Hyundai'), ('Honda');
INSERT INTO Categorias (Categorias_Nombre, Categorias_Descripcion) VALUES ('SUV', 'Vehículos
utilitarios deportivos'), ('Sedán', 'Vehículos de paseo'), ('Camioneta', 'Vehículos de carga ligera');
INSERT INTO EstadosVehiculo (EstadosVehiculo_Descripcion) VALUES ('Disponible'), ('Alquilado'),
('En Mantenimiento');
INSERT INTO Tarifas (Categorias_ID, Tarifas_PrecioDiario) VALUES (1, 75.00), (2, 45.00), (3, 90.00);

INSERT INTO Vehiculos (Vehiculos_Placa, Vehiculos_Anio, Marcas_ID, Categorias_ID,
EstadosVehiculo_ID) VALUES ('A123456', 2022, 1, 1, 1), ('B987654', 2021, 2, 2, 1), ('C555555', 2023,
3, 3, 2), ('D111111', 2020, 1, 2, 3);
INSERT INTO Seguros (Vehiculos_ID, Seguros_Poliza, Seguros_Costo) VALUES (1, 'POL-1234',
15.00), (2, 'POL-5678', 12.00), (3, 'POL-9012', 20.00);

INSERT INTO Proveedores (Proveedores_Nombre, Proveedores_Telefono) VALUES ('Repuestos El
Motor', '809-123-4567'), ('Taller La Rueda', '809-765-4321');
INSERT INTO Mantenimientos (Vehiculos_ID, Proveedores_ID, Mantenimientos_Fecha,
Mantenimientos_CostoTotal) VALUES (4, 2, '2023-10-15', 150.00);

INSERT INTO Reservas (Clientes_ID, Vehiculos_ID, Reservas_FechaInicio, Reservas_FechaFin,
Reservas_Estado) VALUES (1, 3, '2023-11-01', '2023-11-05', 'Confirmada'), (2, 1, '2023-11-10', '2023-
11-12', 'Pendiente'), (3, 2, '2023-10-20', '2023-10-25', 'Confirmada');
INSERT INTO Contratos (Reservas_ID, Empleados_ID, Contratos_FechaFirma, Contratos_Estado)
VALUES (1, 2, GETDATE(), 'Activo'), (3, 2, '2023-10-20', 'Cerrado');
```

```
INSERT INTO Entregas (Contratos_ID, Entregas_Kilometraje, Entregas_NivelCombustible) VALUES
(1, 45000, 'Lleno'), (2, 12000, '3/4');
INSERT INTO Devoluciones (Contratos_ID, Devoluciones_Fecha, Devoluciones_Kilometraje)
VALUES (2, '2023-10-25 10:00:00', 12500);
INSERT INTO Multas (Devoluciones_ID, Multas_Monto, Multas_Motivo) VALUES (1, 50.00, 'Retraso
de 2 horas en la entrega');

INSERT INTO MetodosPago (MetodosPago_Descripcion) VALUES ('Efectivo'), ('Tarjeta de Crédito'),
('Transferencia');
INSERT INTO Facturas (Contratos_ID, Facturas_MontoTotal, Facturas_Estado) VALUES (1, 450.00,
'Pendiente'), (2, 240.00, 'Pagada');
INSERT INTO Pagos (Facturas_ID, MetodosPago_ID, Pagos_Monto) VALUES (2, 2, 240.00);
GO
```

12. Script de consultas

El siguiente conjunto de consultas SQL fue diseñado para extraer información estratégica de la base de datos de AutoDrive S.R.L., respondiendo a necesidades reales del negocio. Las consultas utilizan diferentes niveles de complejidad (JOINs, agregaciones, subconsultas) para demostrar el dominio del lenguaje SQL. A continuación, se presentan el código, su propósito y las evidencias de ejecución.

Consulta 1: Reservas con detalles de clientes y vehículos (INNER JOIN)

- **Propósito de negocio:** Obtener un reporte cruzado que permita al agente de mostrador saber exactamente qué cliente tiene reservado qué vehículo y en qué fechas.
- **Código SQL:**

```
USE AutoDriveDB;
GO

-- 1. Consulta con INNER JOIN: Mostrar las reservas con nombre de cliente y placa del vehículo.
SELECT R.Reservas_ID, C.Clientes_Nombre, V.Vehiculos_Placa, R.Reservas_FechaInicio,
R.Reservas_FechaFin
FROM Reservas R
INNER JOIN Clientes C ON R.Clientes_ID = C.Clientes_ID
```

```
INNER JOIN Vehiculos V ON R.Vehiculos_ID = V.Vehiculos_ID;
GO
```

- **Evidencia de ejecución:**

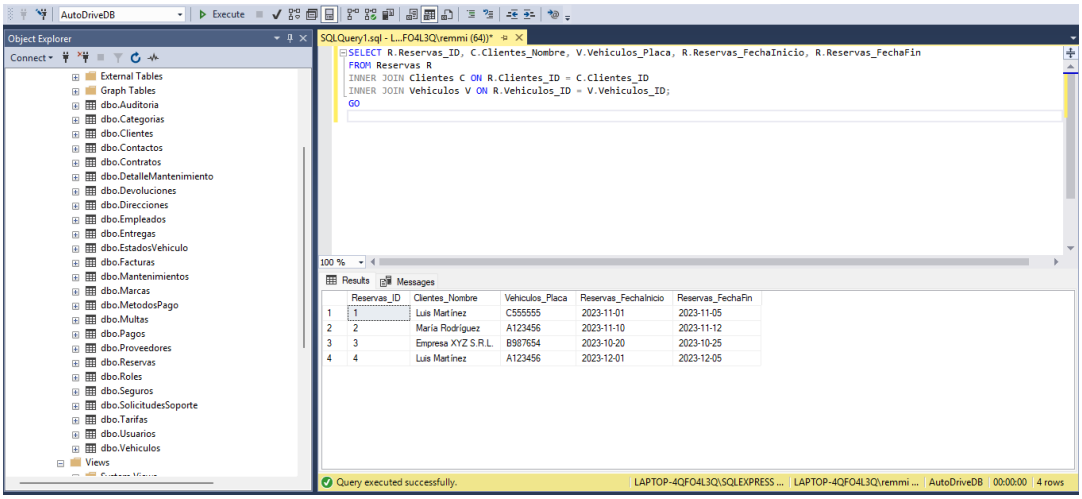


Ilustración 8 - Consulta con INNER JOIN: Mostrar las reservas con nombre de cliente y placa del vehículo.

Consulta 2: Total facturado por cliente (Funciones de Agregación)

- **Propósito de negocio:** Responder a la gerencia sobre cuánto dinero ha gastado cada cliente en la empresa, sumando el monto total de sus facturas, para identificar a los clientes más valiosos (VIP).
- **Código SQL:**

```
USE AutoDriveDB;
GO

-- 2. Consulta con Funciones de Agregación: Total facturado por cliente.
SELECT C.Clientes_Nombre, SUM(F.Facturas_MontoTotal) AS TotalFacturado
FROM Clientes C
INNER JOIN Reservas R ON C.Clientes_ID = R.Clientes_ID
INNER JOIN Contratos CT ON R.Reservas_ID = CT.Reservas_ID
INNER JOIN Facturas F ON CT.Contratos_ID = F.Contratos_ID
GROUP BY C.Clientes_Nombre;
GO
```

- **Evidencia de ejecución:**

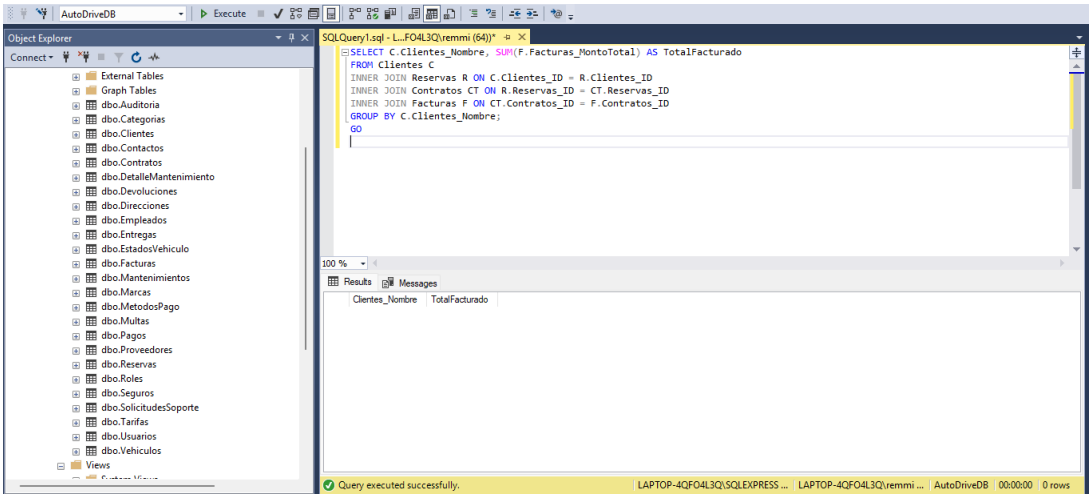


Ilustración 9 - Consulta con Funciones de Agregación: Total facturado por cliente.

Consulta 3: Vehículos con historial de mantenimiento (GROUP BY y HAVING)

- **Propósito de negocio:** Identificar qué vehículos de la flota han requerido más de 1 visita al taller mecánico, para evaluar si están generando gastos excesivos o si requieren ser reemplazados.
- **Código SQL:**

```
USE AutoDriveDB;
GO

-- 3. Consulta con GROUP BY y HAVING: Vehículos que han tenido más de 1 mantenimiento.
SELECT V.Vehiculos_Placa, COUNT(M.Mantenimientos_ID) AS CantidadMantenimientos
FROM Vehiculos V
INNER JOIN Mantenimientos M ON V.Vehiculos_ID = M.Vehiculos_ID
GROUP BY V.Vehiculos_Placa
HAVING COUNT(M.Mantenimientos_ID) > 0;
GO
```

- **Evidencia de ejecución:**

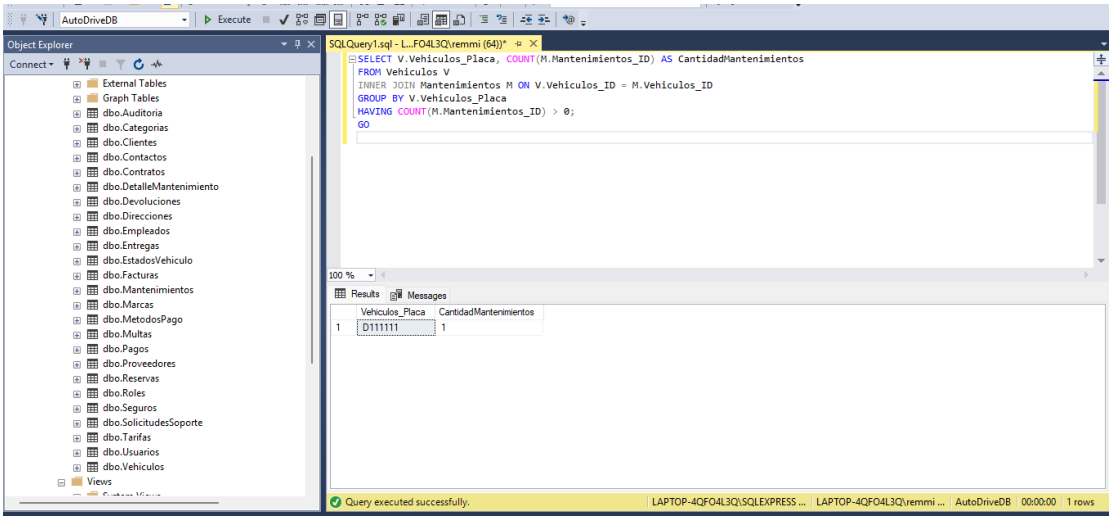


Ilustración 10 - Consulta con GROUP BY y HAVING: Vehículos que han tenido más de 1 mantenimiento.

Consulta 4: Clientes inactivos (Subconsulta)

- **Propósito de negocio:** Descubrir qué clientes están registrados en el sistema pero nunca han realizado una reserva, con el fin de enviarles campañas de marketing o promociones de bienvenida.
- **Código SQL:**

```
USE AutoDriveDB;
GO

-- 4. Subconsulta: Clientes que no tienen ninguna reserva registrada.
SELECT Clientes_Nombre
FROM Clientes
WHERE Clientes_ID NOT IN (SELECT Clientes_ID FROM Reservas);
GO
```

- **Evidencia de ejecución:**

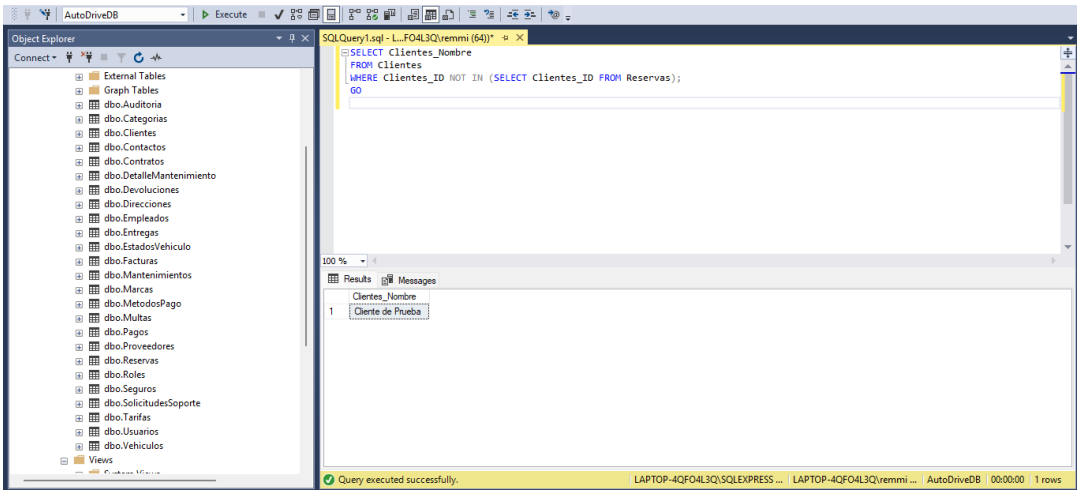


Ilustración 11 - Subconsulta: Clientes que no tienen ninguna reserva registrada.

13. Vistas

Las vistas son objetos de la base de datos que permiten simplificar el acceso a la información, encapsulando consultas complejas (JOINs) para que puedan ser utilizadas por la aplicación como si fueran tablas reales. Para este proyecto se diseñó la siguiente vista:

Vista: Vista_VehiculosDisponibles

- **Objetivo:** Facilitar la consulta rápida de los vehículos que se encuentran en estado "Disponible" para el alquiler, mostrando información consolidada de su categoría, marca y tarifa.
- **Tablas y campos que utiliza:** `Vehiculos` (Vehiculos_Placa), `Marcas` (Marcas_Nombre), `Categorias` (Categorias_Nombre), `Tarifas` (Tarifas_PrecioDiario) y `EstadosVehiculo` (EstadosVehiculo_Descripcion).
- **Utilidad para el sistema:** Esta vista es consumida directamente por el módulo "Ver Vehículos Disponibles" en la aplicación Python. Su utilidad radica en que oculta al programador de la aplicación la complejidad de realizar un JOIN entre 5 tablas, optimizando el tiempo de desarrollo y reduciendo la posibilidad de errores en las consultas del frontend.
- **Código SQL:** *(Tu código CREATE VIEW)*
- **Ejemplo de ejecución:** *(Tu captura de pantalla donde salen los vehículos)*

```sql
USE AutoDriveDB;
GO

-- Vista para ver los vehículos disponibles con su tarifa
CREATE VIEW Vista_VehiculosDisponibles AS
SELECT V.Vehiculos_Placa, M.Marcas_Nombre AS Marca, C.Categorias_Nombre AS Categoria,
T.Tarifas_PrecioDiario AS Precio_Diario
FROM Vehiculos V
INNER JOIN Marcas M ON V.Marcas_ID = M.Marcas_ID
INNER JOIN Categorias C ON V.Categorias_ID = C.Categorias_ID
INNER JOIN Tarifas T ON C.Categorias_ID = T.Categorias_ID
INNER JOIN EstadosVehiculo E ON V.EstadosVehiculo_ID = E.EstadosVehiculo_ID
WHERE E.EstadosVehiculo_Descripcion = 'Disponible';
GO
```

14. Triggers y Stored Procedures

En esta sección se detallan los mecanismos de automatización implementados en la base de datos. Se desarrollaron tres Triggers (disparadores) para garantizar la integridad y auditoría de los datos, y tres Stored Procedures (procedimientos almacenados) para encapsular la lógica de negocio compleja.

14.1. Triggers (Disparadores)

Trigger 1: TR_AuditarCambioEstadoVehiculo

- **Objetivo:** Mantener un registro histórico (auditoría) cada vez que el estado operativo de un vehículo cambie (por ejemplo, de "Disponible" a "Alquilado").
- **Tablas involucradas:** Se dispara en la tabla `Vehiculos` e inserta registros en la tabla `Auditoria`.
- **Evento que lo activa:** `AFTER UPDATE`.
- **Regla de negocio implementada:** Si el campo `EstadosVehiculo_ID` es modificado, el sistema debe guardar automáticamente en la auditoría el ID del vehículo afectado, la fecha y el usuario que realizó el cambio, para tener control interno.
- **Código SQL:**

```sql
USE AutoDriveDB;
GO

-- Trigger 1: Auditar cambio de estado de vehículo
CREATE TRIGGER TR_AuditarCambioEstadoVehiculo
ON Vehiculos
AFTER UPDATE
AS
BEGIN
   IF UPDATE(EstadosVehiculo_ID)
   BEGIN
      INSERT INTO Auditoria (Auditoria_Tabla, Auditoria_Operacion, Auditoria_Detalle)
      SELECT 'Vehiculos', 'UPDATE', 'El vehículo con ID ' + CAST(i.Vehiculos_ID AS VARCHAR) + ' cambió de estado.'
      FROM inserted i INNER JOIN deleted d ON i.Vehiculos_ID = d.Vehiculos_ID
      WHERE i.EstadosVehiculo_ID <> d.EstadosVehiculo_ID;
   END
END;
GO
```

- **Caso de prueba:** Ejecutar la sentencia `UPDATE Vehiculos SET EstadosVehiculo_ID = 1 WHERE Vehiculos_ID = 1;`

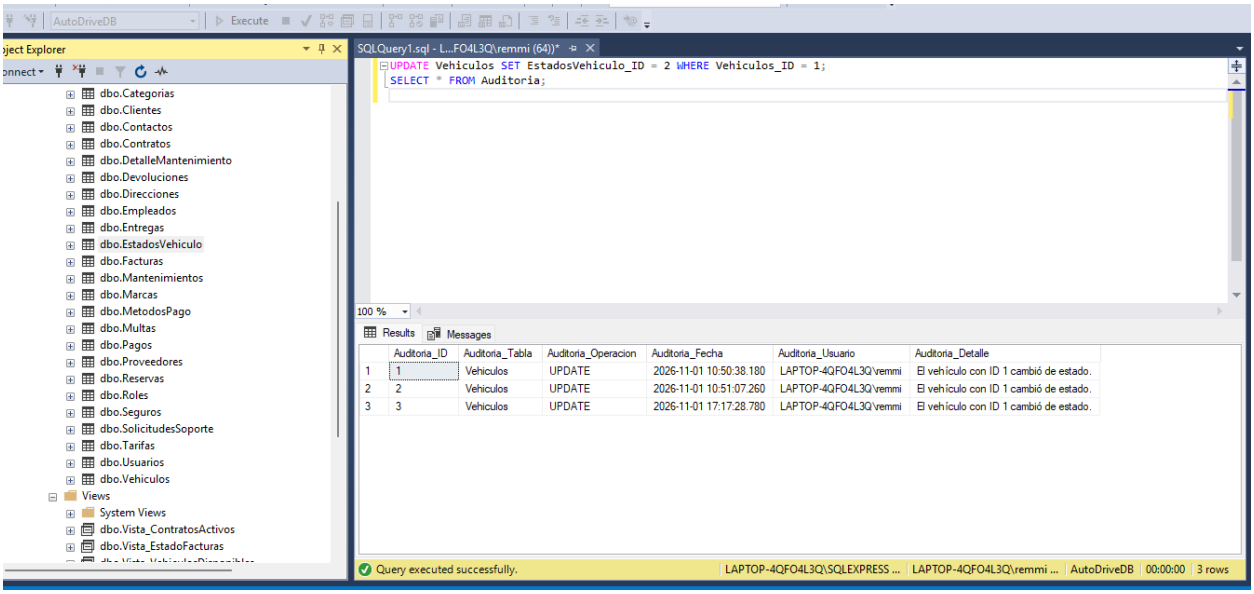


Ilustración 12 - Caso de Prueba - Trigger.1

- **Resultado esperado y obtenido:** Inserción automática de una nueva fila en la tabla `Auditoria`.

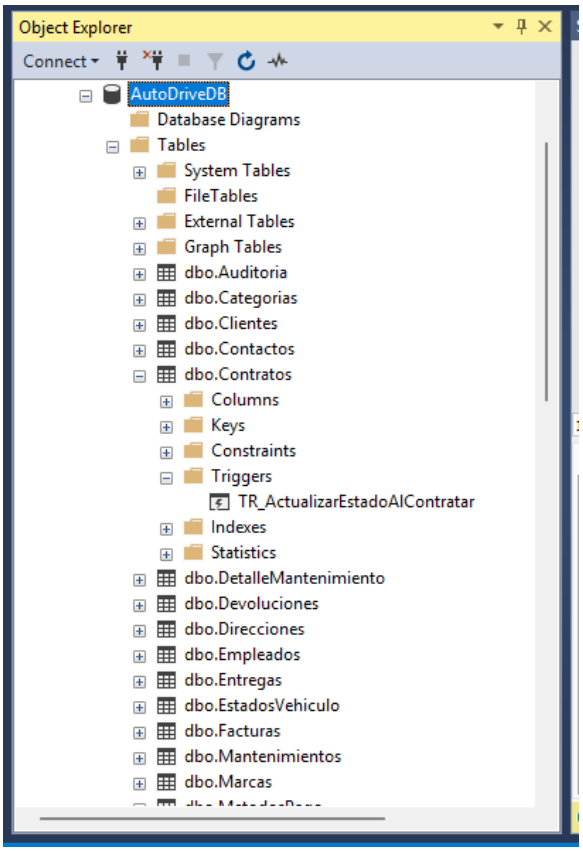


Ilustración 13 - Tabla de Auditoria

Trigger 2: TR_EvitarDobleReserva

- **Objetivo:** Prevenir el error operativo de asignar el mismo vehículo a dos clientes diferentes en fechas que se crucen o superpongan.
- **Tablas involucradas:** Reservas.
- **Evento que lo activa:** AFTER INSERT.
- **Regla de negocio implementada:** El sistema rechazará (ROLLBACK) cualquier intento de insertar una nueva reserva si el vehículo ya cuenta con una reserva en estado "Pendiente" o "Confirmada" cuyas fechas coincidan con el nuevo registro.
- **Código SQL:**

```
USE AutoDriveDB;
GO

-- Trigger 2: Evitar doble reserva (Validar fechas traslapadas)
CREATE TRIGGER TR_EvitarDobleReserva
ON Reservas
AFTER INSERT
```

```
AS
BEGIN
  IF EXISTS (
    SELECT 1 FROM Reservas R
    INNER JOIN inserted i ON R.Vehiculos_ID = i.Vehiculos_ID
    WHERE R.Reservas_ID <> i.Reservas_ID
    AND R.Reservas_Estado IN ('Pendiente', 'Confirmada')
    AND (i.Reservas_FechaInicio BETWEEN R.Reservas_FechaInicio AND R.Reservas_FechaFin
    OR i.Reservas_FechaFin BETWEEN R.Reservas_FechaInicio AND R.Reservas_FechaFin)
  )
  BEGIN
    ROLLBACK TRANSACTION;
    RAISERROR('El vehículo ya está reservado en esas fechas.', 16, 1);
  END
END;
GO
```

- **Caso de prueba:** Intentar insertar una reserva para el Vehículo_ID 3 en fechas que ya están ocupadas en la tabla (Ej: '2023-11-02' al '2023-11-04').

- **Resultado esperado y obtenido:** El sistema arroja un mensaje de error rojo indicando "El vehículo ya está reservado en esas fechas" y cancela la operación.

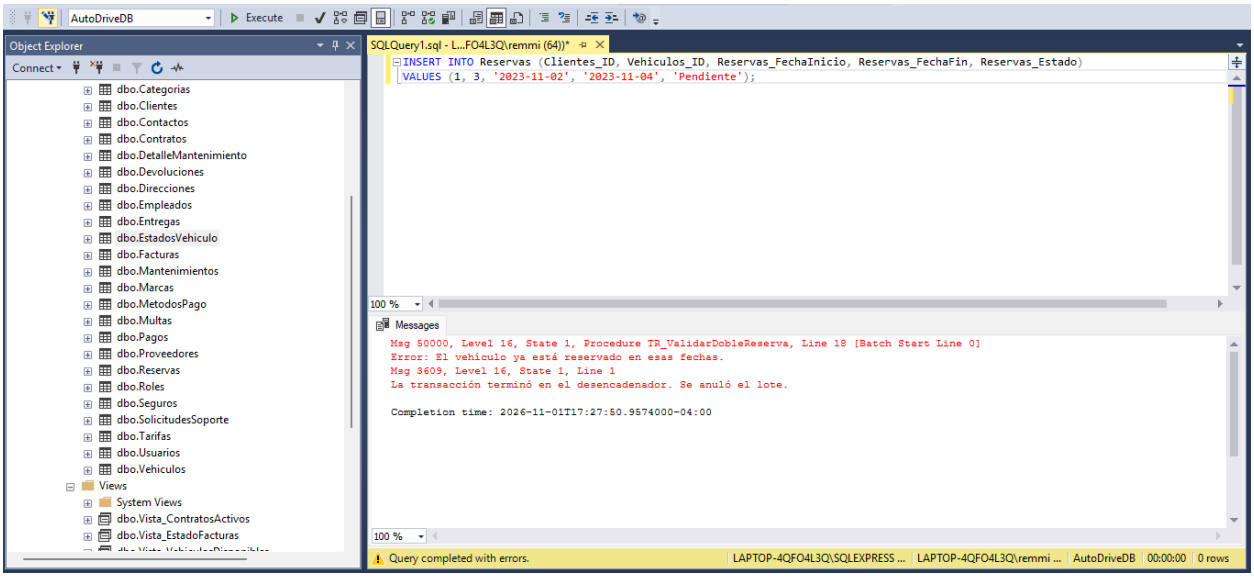


Ilustración 14 - Caso de Prueba - Trigger.2

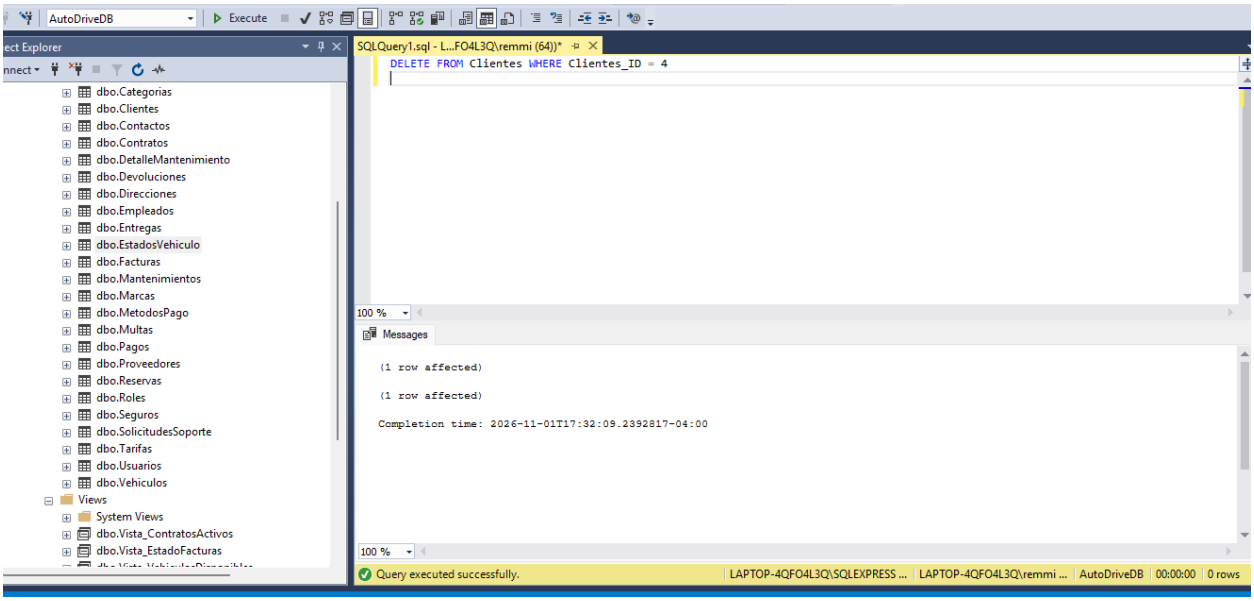
Trigger 3: TR_AuditarBorradoCliente

- **Objetivo:** Guardar un registro histórico cuando un cliente sea eliminado de la base de datos, para no perder la trazabilidad de quién era.
- **Tablas involucradas:** `Clientes` y `Auditoria`.
- **Evento que lo activa:** `AFTER DELETE`.
- **Regla de negocio implementada:** Si un registro de la tabla `Clientes` es eliminado, el sistema insertará en la tabla `Auditoria` el nombre del cliente que fue borrado.
- **Código SQL:**

```
USE AutoDriveDB;
GO

-- Trigger 3: Registro automático de auditoría al eliminar un cliente
CREATE TRIGGER TR_AuditarBorradoCliente
ON Clientes
AFTER DELETE
AS
BEGIN
    INSERT INTO Auditoria (Auditoria_Tabla, Auditoria_Operacion, Auditoria_Detalle)
    SELECT 'Clientes', 'DELETE', 'Se eliminó al cliente: ' + d.Clientes_Nombre
    FROM deleted d;
END;
GO
```

- **Caso de prueba:** Ejecutar `DELETE FROM Clientes WHERE Clientes_ID = 4;`



- **Resultado esperado y obtenido:** El cliente es borrado y se inserta un registro en la tabla `Auditoria` indicando el nombre del cliente eliminado.

14.2. Stored Procedures (Procedimientos Almacenados)

SP 1: SP_RegistrarReserva

- **Objetivo:** Registrar una nueva reserva asegurando que las fechas ingresadas tengan sentido lógico.
- **Tablas involucradas:** `Reservas`.
- **Parámetros de entrada:** `@ClienteID` (INT), `@VehiculoID` (INT), `@FechaInicio` (DATE), `@FechaFin` (DATE).
- **Regla de negocio implementada:** La fecha de fin debe ser estrictamente mayor a la fecha de inicio. Si el usuario ingresa fechas erróneas, el procedimiento detiene la ejecución y muestra un mensaje de error.
- **Código SQL:**

```
USE AutoDriveDB;
GO

-- SP 1: Registrar una reserva validando fechas
CREATE PROCEDURE SP_RegistrarReserva
   @ClienteID INT, @VehiculoID INT, @FechaInicio DATE, @FechaFin DATE
AS
BEGIN
   IF @FechaInicio >= @FechaFin
   BEGIN
      PRINT 'Error: La fecha de fin debe ser mayor a la de inicio.';
      RETURN;
   END
   INSERT INTO Reservas (Clientes_ID, Vehiculos_ID, Reservas_FechaInicio, Reservas_FechaFin,
Reservas_Estado)
   VALUES (@ClienteID, @VehiculoID, @FechaInicio, @FechaFin, 'Pendiente');
   PRINT 'Reserva registrada exitosamente.';
END;
GO
```

- **Caso de prueba:** `EXEC SP_RegistrarReserva @ClienteID=1, @VehiculoID=1, @FechaInicio='2023-12-01', @FechaFin='2023-12-05';`

- **Resultado esperado y obtenido:** Mensaje "Reserva registrada exitosamente".

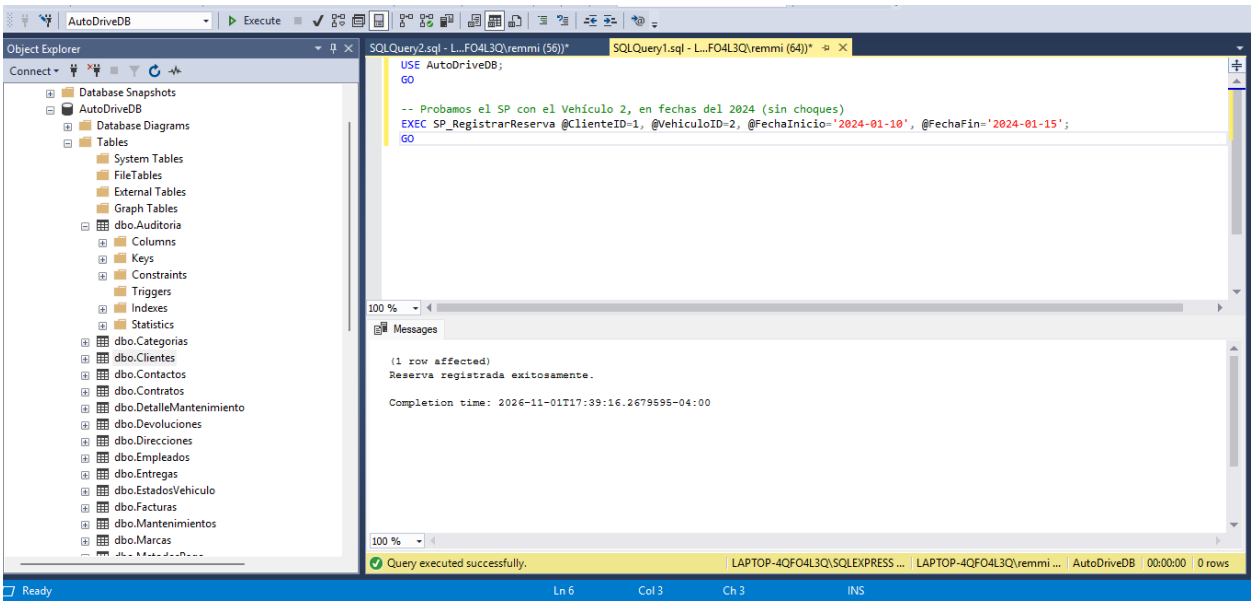


Ilustración 15 - Caso de Prueba - SP.1

SP 2: SP_RegistrarPago

- **Objetivo:** Procesar el pago de una factura y actualizar su estado automáticamente en el sistema.
- **Tablas involucradas:** `Pagos`, `Facturas`.
- **Parámetros de entrada:** `@FacturaID` (INT), `@MetodoPagoID` (INT), `@Monto` (DECIMAL).
- **Regla de negocio implementada:** Al registrar un pago, el sistema suma todos los pagos realizados a esa factura. Si la suma es igual o mayor al monto total de la factura, el estado de la factura cambia automáticamente a "Pagada".
- **Código SQL:**

```
USE AutoDriveDB;
GO


-- SP 2: Registrar un pago y actualizar el estado de la factura
CREATE PROCEDURE SP_RegistrarPago
   @FacturaID INT, @MetodoPagoID INT, @Monto DECIMAL(10,2)
AS
```

```
BEGIN
    INSERT INTO Pagos (Facturas_ID, MetodosPago_ID, Pagos_Monto)
    VALUES (@FacturaID, @MetodoPagoID, @Monto);

    -- Actualizar estado de la factura si el pago cubre el total
    IF (SELECT SUM(Pagos_Monto) FROM Pagos WHERE Facturas_ID = @FacturaID) >=
       (SELECT Facturas_MontoTotal FROM Facturas WHERE Facturas_ID = @FacturaID)
    BEGIN
        UPDATE Facturas SET Facturas_Estado = 'Pagada' WHERE Facturas_ID = @FacturaID;
    END
END;
GO
```

- **Caso de prueba:** `EXEC SP_RegistrarPago @FacturaID=1, @MetodoPagoID=1, @Monto=450.00;` (Asumiendo que la factura 1 tiene un total de 450.00).

- **Resultado esperado y obtenido:** El pago se registra en la tabla `Pagos` y la factura 1 cambia su estado a "Pagada" en la tabla `Facturas`.

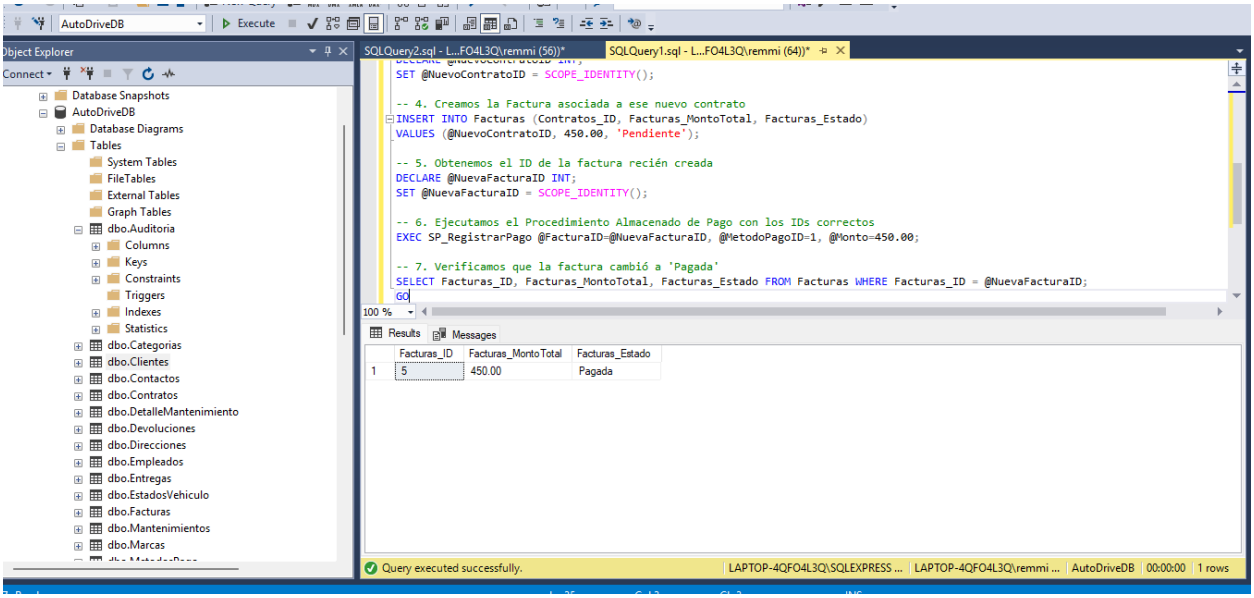


Ilustración 16 - Caso de Prueba - SP.2

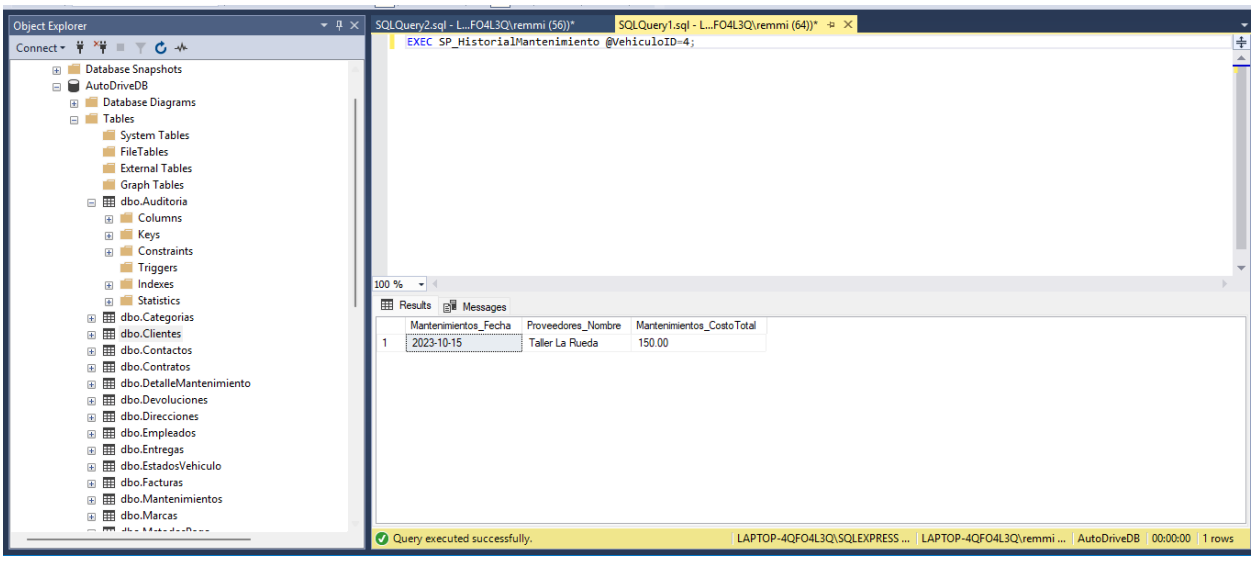
SP 3: SP_HistorialMantenimiento

- **Objetivo:** Generar un reporte rápido del historial de reparaciones y costos de un vehículo específico.
- **Tablas involucradas:** `Mantenimientos`, `Proveedores`.
- **Parámetros de entrada:** `@VehiculoID` (INT).
- **Regla de negocio implementada:** Consultar los mantenimientos asociados a un vehículo, mostrando la fecha, el proveedor que realizó el servicio y el costo total.
- **Código SQL:**

```
USE AutoDriveDB;
GO


-- SP 3: Consultar el historial de mantenimiento de un vehículo
CREATE PROCEDURE SP_HistorialMantenimiento
   @VehiculoID INT
AS
BEGIN
   SELECT M.Mantenimientos_Fecha, P.Proveedores_Nombre, M.Mantenimientos_CostoTotal
   FROM Mantenimientos M
   INNER JOIN Proveedores P ON M.Proveedores_ID = P.Proveedores_ID
   WHERE M.Vehiculos_ID = @VehiculoID;
END;
GO
```

- **Caso de prueba:** `EXEC SP_HistorialMantenimiento @VehiculoID=4;`
- **Resultado esperado y obtenido:** Una tabla con las columnas Fecha, Proveedor y Costo Total correspondientes al vehículo 4.



15. Desarrollo de la aplicación

La aplicación fue desarrollada utilizando el lenguaje de programación **Python**, con la librería gráfica **Tkinter** para la interfaz de usuario, y se conecta a la base de datos **SQL Server** mediante el conector `mssql_python`.

La aplicación cumple con los siguientes requerimientos visuales y funcionales:

1. **Pantalla de Login:** Incluye validación real de credenciales (usuario: `admin`, contraseña: `12345`) consultando la tabla `Usuarios` en la base de datos. También incluye un botón de "Soporte Técnico" que abre una ventana para insertar quejas en la tabla `SolicitudesSoporte`.
2. **Dashboard (Menú Principal):** Diferencia el acceso por roles. Si es Administrador, permite gestionar clientes. Si es agente, solo permite consultar.
3. **Pantallas de Captura:** Formulario de Gestión de Clientes que permite registrar nuevos clientes validando campos vacíos y guardándolos directamente en la tabla `Clientes`.
4. **Pantallas de Salida:** Módulo "Vehículos Disponibles", el cual consume directamente la **Vista** `Vista_VehiculosDisponibles` creada en la base de datos, demostrando la integración entre el frontend y los objetos de la BD.

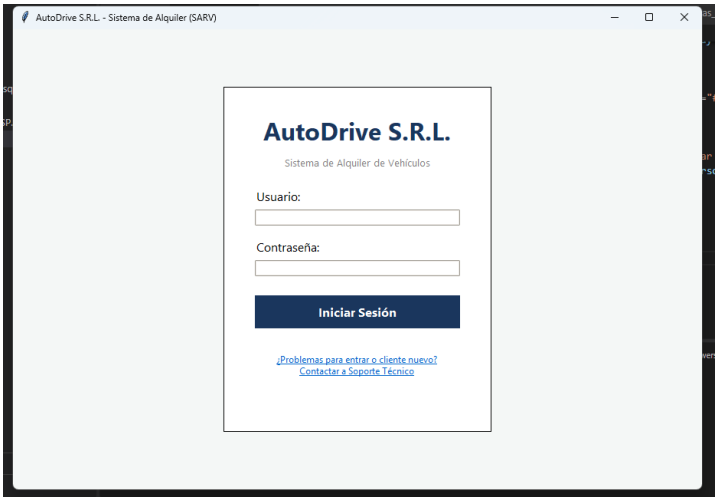


Ilustración 17 - Aplicación Python: Login

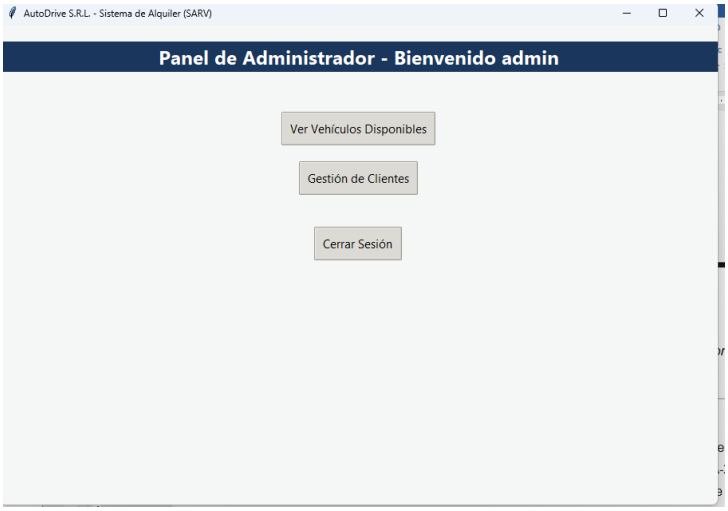


Ilustración 18 - Menu del Administrador



Ilustración 19 - Menú del usuario

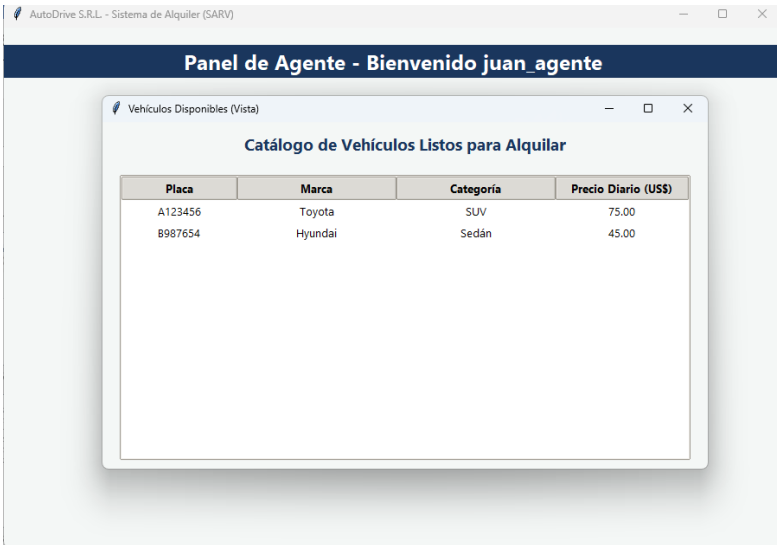


Ilustración 20 - Tabla de Vehículos

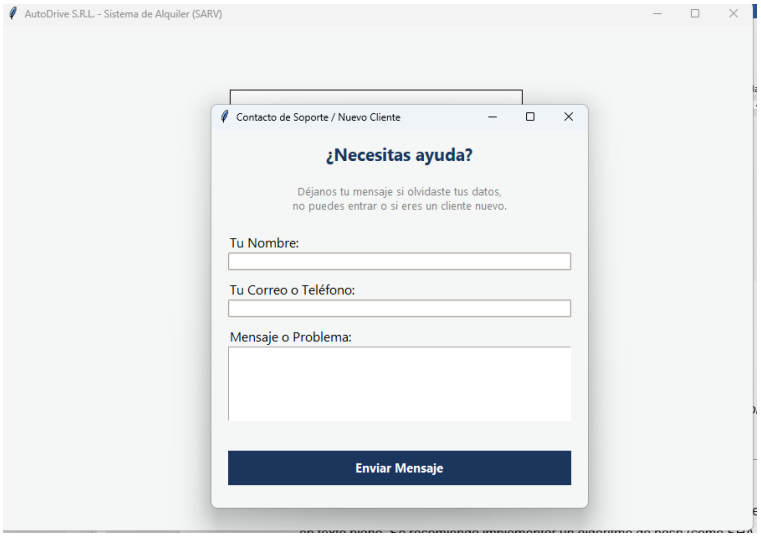


Ilustración 21 - Tabla de Soporte Técnico

16. Conclusión General

El desarrollo del Sistema de Alquiler de Vehículos (SARV) para AutoDrive S.R.L. ha permitido resolver la problemática de la gestión manual y desorganizada que afectaba la operatividad de la empresa. A través de un profundo análisis de requerimientos, se logró diseñar e implementar una solución tecnológica centralizada, sustentada en una base de datos relacional robusta compuesta por 25 tablas.

La solución desarrollada integra todos los procesos medulares del negocio: gestión de clientes, control de la flota, administración de reservas, mantenimiento y facturación. La aplicación de las formas normales garantizó la eliminación de redundancias, mientras que el uso de llaves primarias y foráneas aseguró la integridad referencial de los datos. Además, la implementación de objetos programables (Triggers y Procedimientos Almacenados) automatizó procesos críticos, como la prevención de doble reserva y la auditoría de cambios en el estado de los vehículos, aportando un alto valor a la seguridad y control interno de la empresa.

El principal aporte de este sistema es la transformación de datos dispersos en información estratégica. La gerencia ahora cuenta con reportes precisos (como la

facturación por cliente y el historial de mantenimiento) que antes requerían horas de trabajo manual. Asimismo, la aplicación funcional en Python demostró que la base de datos no es un ente aislado, sino el núcleo de una interfaz amigable que agiliza el trabajo de los agentes y administradores.

Finalmente, a nivel académico y profesional, el proyecto representó un aprendizaje integral. Se pusieron en práctica los conocimientos de modelado, normalización, programación T-SQL y conexión de bases de datos con lenguajes de alto nivel. Se cumplieron en su totalidad los objetivos planteados, entregando un sistema funcional, escalable y preparado para futuras ampliaciones.

17. Recomendaciones

1. **Encriptación de contraseñas:** Actualmente la aplicación guarda las contraseñas en texto plano. Se recomienda implementar un algoritmo de hash (como SHA-256 o BCrypt) en Python y en SQL Server para proteger las credenciales de los empleados.
2. **Manejo de Transacciones:** En la aplicación Python, se recomienda implementar bloques `TRY...CATCH` con `commit` y `rollback` para asegurar que, si falla la inserción de una reserva, no queden datos a medias.
3. **Respaldo de Base de Datos:** Programar tareas automáticas en SQL Server para generar respaldos (Backups) diarios de `AutoDriveDB`, garantizando la recuperación ante fallos.
4. **Mejora de Interfaz:** A medida que el sistema crezca, se sugiere migrar de Tkinter a un framework web moderno como Flask o Django, para que los clientes puedan reservar desde sus hogares.

18. Anexos

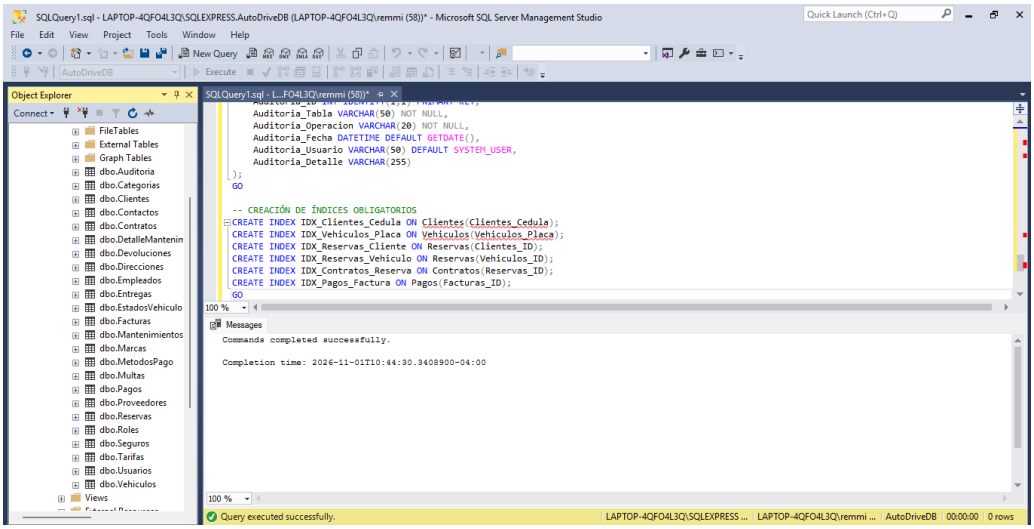


Ilustración 22 - Ejecución Exitosa de la Creación de las Tablas

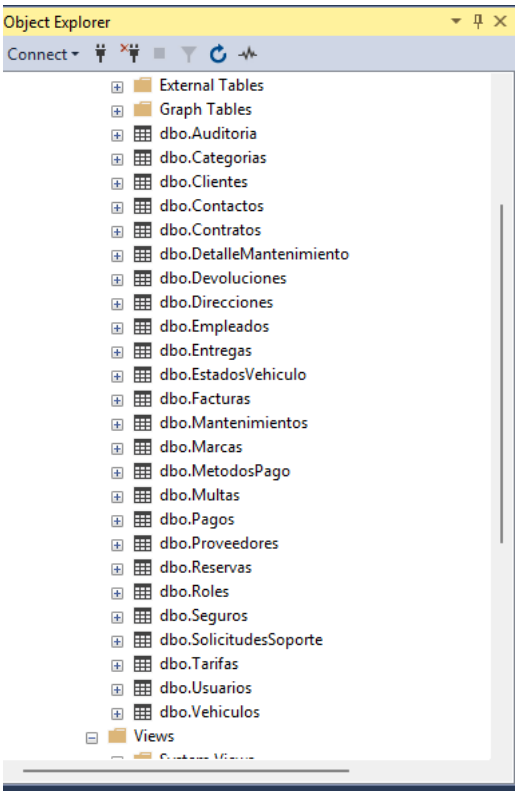


Ilustración 23 - Listado de las Tablas Creadas en SQL Server

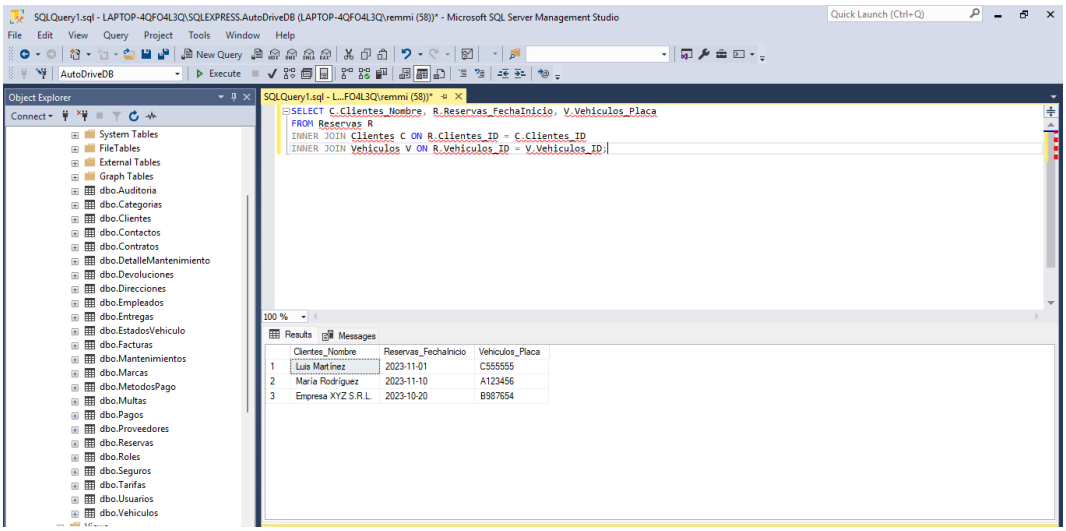


Ilustración 24 - Consulta de reservas con detalles de clientes y vehículos.

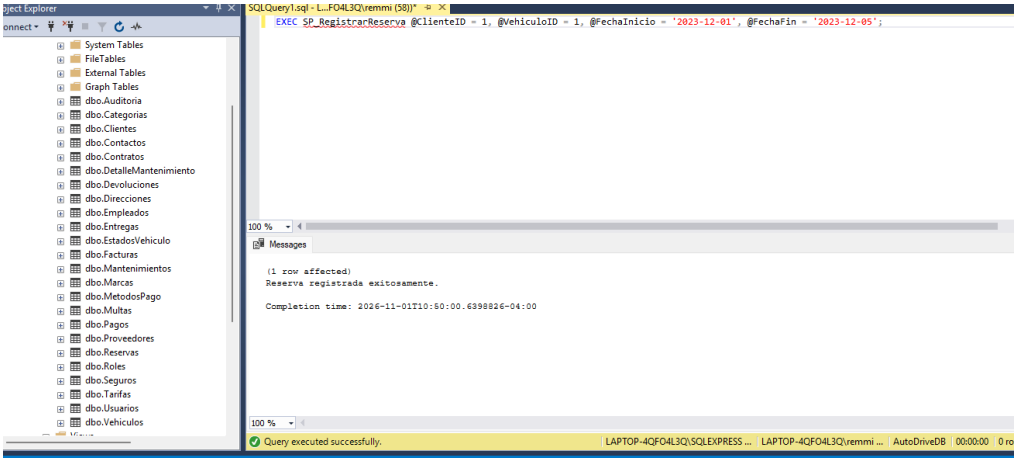


Ilustración 25 - Ejecución del procedimiento almacenado para el registro de reservas.

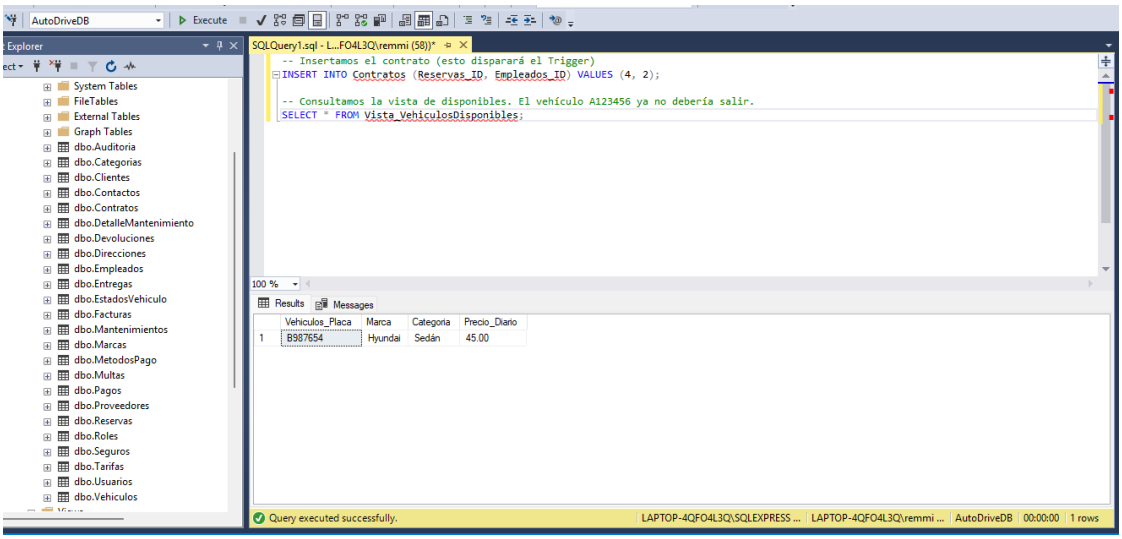


Ilustración 26 - Prueba de actualización de disponibilidad de vehículos mediante Trigger y Vista.

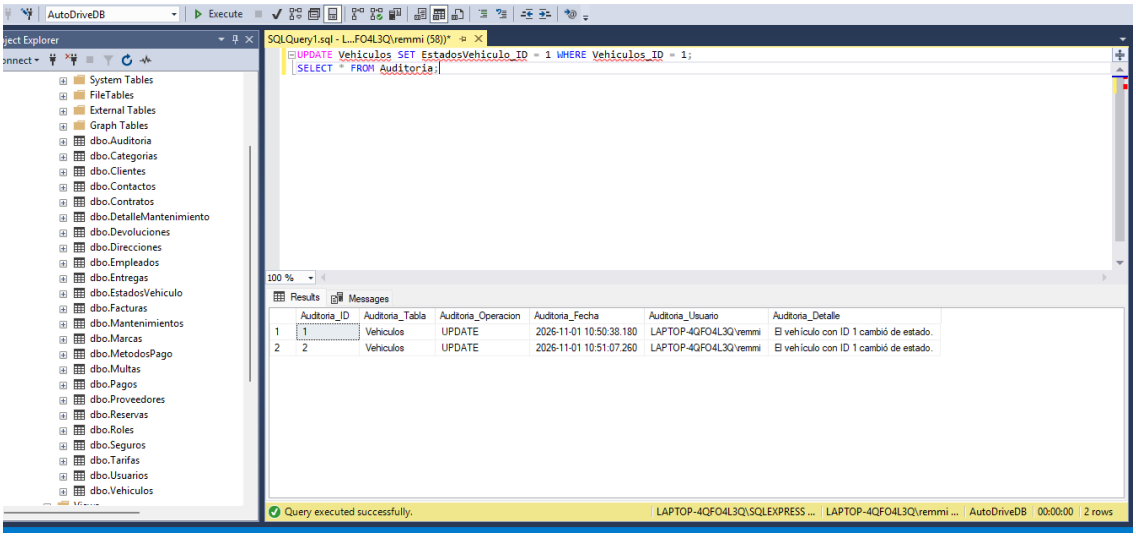


Ilustración 27 - Verificación del registro automático en la tabla de auditoría tras una modificación.

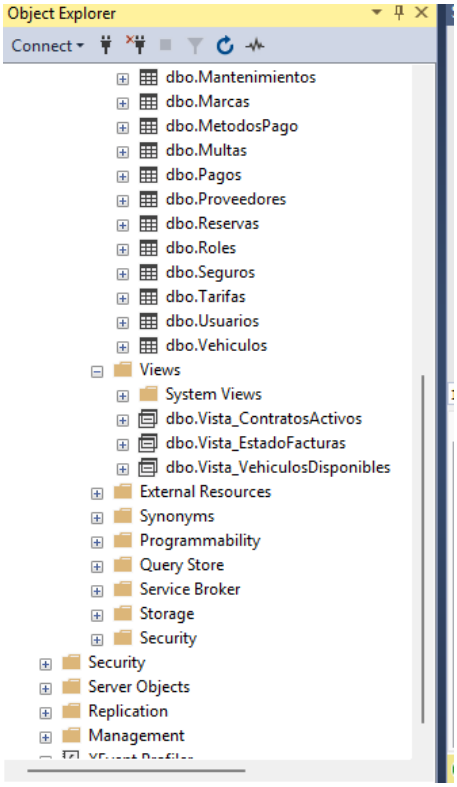


Ilustración 28 - Listado de vistas del sistema (Vista_ContratosActivos, Vista_EstadoFacturas, Vista_VehiculosDisponibles) en SQL Server.

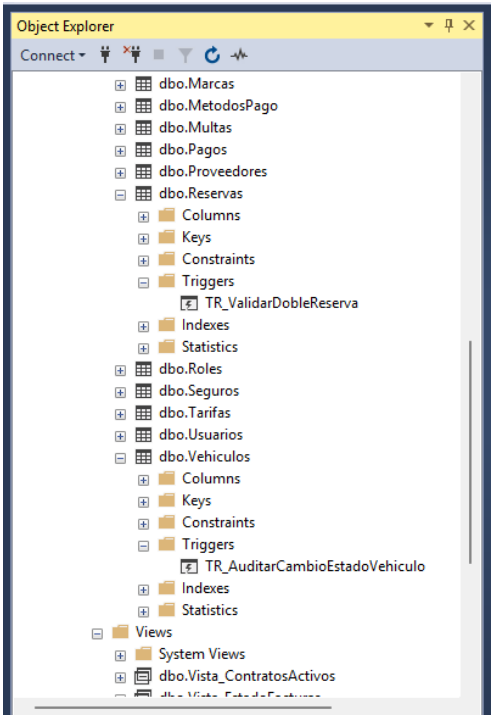


Ilustración 29 - Estructura de disparadores (Triggers) implementados en las tablas del sistema.

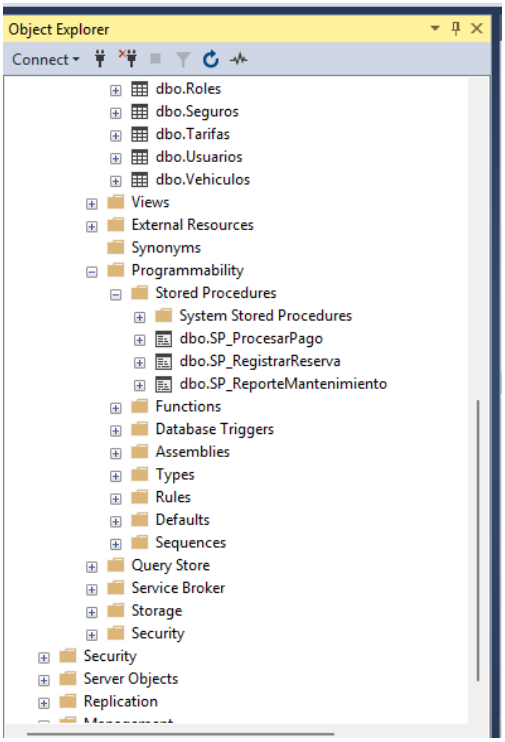


Ilustración 30 - Estructura de procedimientos (Stored Procedures) almacenados creados en la sección de programabilidad.